Can This Agent Even Do That? A Decidable Goal-Achievability Type Discipline for LLM-Synthesized Agent Skills

Tzu-Chun Chen
Microsoft, ISD EAG
tzuchunchen@microsoft.com

Mariangiola Dezani-Ciancaglini
Department of Computer Science
University of Turin
dezani@di.unito.it

Abstract

Modern autonomous agents rely on natural-language skills that specify tools, workflows, and desired outcomes. Such skills may be written by human experts or generated with LLM assistance, but their goals may be structurally impossible to achieve: a required tool may be unavailable, a constraint may be unsatisfiable, or agents may deadlock. We present a *skill compiler* that checks goal achievability before execution, without relying on an LLM judge. It combines capability-based reachability from automated planning with direct checking of multiparty communication protocols [1, 2]. An LLM may compact a skill into a formal pack of capabilities, preconditions, effects, goals, and protocol structure, which is then checked deterministically. A Coq-verified abstraction theorem, together with a mechanized lemma that ties it to the checker's symbolic search, guarantees that an *impossible* verdict is correct for the formal pack being checked. We evaluate it on every real use case we have run. With the pack read from the environment, it gave the correct verdict in 100% over 21 pre-flight runs in a real workspace, with no model tokens, and in 100% over 123 checks in Microsoft Agent 365 workflows. On 447 real public skills compacted by an LLM and labelled by blind execution, it was right in 88%, caught 84% over 145 blocked skills and wrongly refuted 10% over 302 working ones, close to a direct LLM verdict that gives no witness. Where what blocks a file is rarely written in it, recall falls to 39% over 18 blocked files, against 89% for the direct verdict, with the same rate of wrongly refuted working files (24% over 82). As a gate around a hosted agent it prevented every unapproved send (0% over 21 runs, against 10% ungated). The measured token saving is 1.58×, far below the 87× that our earlier cost model predicted. The check is cheap, explained, and safe in the refutation direction; its weak point is recall, which depends on how completely the compaction states what a skill needs.

1 Introduction

A modern agent is no longer merely a chatbot, but an autonomous system capable of taking actions beyond text generation—for example, writing code, sending emails, or executing trades. Such behaviour is often guided by natural-language artifacts that specify instructions, workflows, and constraints for the agent [3, 4, 6, 7]. A *skill* is one common way of expressing such guidance. It is a folder containing instructions and tooling that an agent can discover and load as needed [5], thereby *harnessing* a general-purpose agent toward a particular outcome.

There is growing evidence that this guidance matters. Across seven state-of-the-art open-source multi-agent systems, failure rates range from 41% to 86.7% [8]. More importantly, some of these failures can be addressed without changing the underlying model. In one system, simply adding a

Preprint.

task-objective verification step to the specification improves task success by 15.6% [8, Appendix H]. Likewise, repairing the harness based on execution traces improves task completion by 6.3 to 18.4 percentage points across four agent benchmarks [4]. These results show that the instructions and structure surrounding an agent can matter as much as the model itself.

The question, however, is how we can know whether such guidance is sufficient to achieve the outcome it declares. A plan may *book a flight and email a confirmation* even though no email tool is available. It may pursue *a flight under $500* even though the available booking tool can return only premium fares. Or a planner may wait for a worker's result while the worker waits for the planner's answer, with each expecting a message that the other will never send. These are not merely execution failures. In each case, the goal is already unreachable before execution begins.

Such failures are structural and recurring, yet today the only way to discover them is often to run the skill and inspect what went wrong afterwards [4]. By then, the agent may already have spent substantial time and tokens taking actions, and a multi-agent system may already have become stuck waiting for communication that will never occur. A simple static check could, in principle, identify these problems before execution.

This paper asks a deliberately narrow question: **given a skill, can its declared goal be achieved at all?** We answer this question with a deterministic formal typing system. We do not verify that every step is safe or that the agent will behave correctly on every possible input. Instead, we check *achievability*. The check is tolerant of small details, detours, and payload variation, which should not by themselves cause a false alarm. At the same time, it is *sound for refutation*: when the compiler reports that a goal is impossible, that verdict is correct.

1.1 The idea, and the trust boundary

The architecture, shown in Figure 1, has two parts separated by a trust boundary. First, an LLM performs *compaction*: it reads the natural-language skill and turns it into a formal *pack* containing capabilities with preconditions and effects, a goal-marked global protocol, and an initial state. This step is *untrusted*: the LLM may misunderstand the skill or omit relevant details.

A deterministic *checker* then decides achievability using only the resulting pack. The checker is mechanically proved sound: if it returns *impossible*, then the goal is impossible under the formal pack it received. If the LLM omits a constraint or capability from the original skill, the checker cannot recover information that is not present in the pack; it will instead reason about the incomplete pack. Conversely, if the LLM introduces a constraint that was not in the original skill, the checker may report the resulting goal as impossible. Thus, our guarantee is about the checker's verdict with respect to the formal pack, not about whether the pack perfectly represents the original natural-language skill.

This does not make the compaction step a weakness that we simply ignore. The formal pack provides a structured target for the LLM to extract. Rather than asking the LLM to freely invent a formal model, we give it a predefined logical structure: capabilities have preconditions and effects, goals are explicit, and the protocol follows a fixed syntax. This makes compaction a constrained translation task and gives us a concrete artifact that can be inspected before execution.

Crucially, the gap between *what the user meant* and *what was formalized* cannot be eliminated. We instead make this gap explicit at the compaction step, where the resulting pack can be inspected and checked before execution. When the compaction is accurate, the checker can identify goals that are impossible before the agent spends time and tokens executing the skill. When the compaction is inaccurate, the checker still provides a sound verdict for the formal pack it received. The key design idea is therefore to move achievability checking from runtime execution to an explicit, inspectable checkpoint before execution.

1.2 Contributions

1. **A goal-achievability type discipline** (Section 3 and Appendices A–C) that combines capability-guarded reachability from automated planning with deadlock-freedom from multiparty session types. The discipline checks whole session configurations directly against a goal-marked global type synthesized from the skill. Building on the multiparty session type tradition of Honda, Yoshida, and Carbone [12, 13], and the direct session-checking

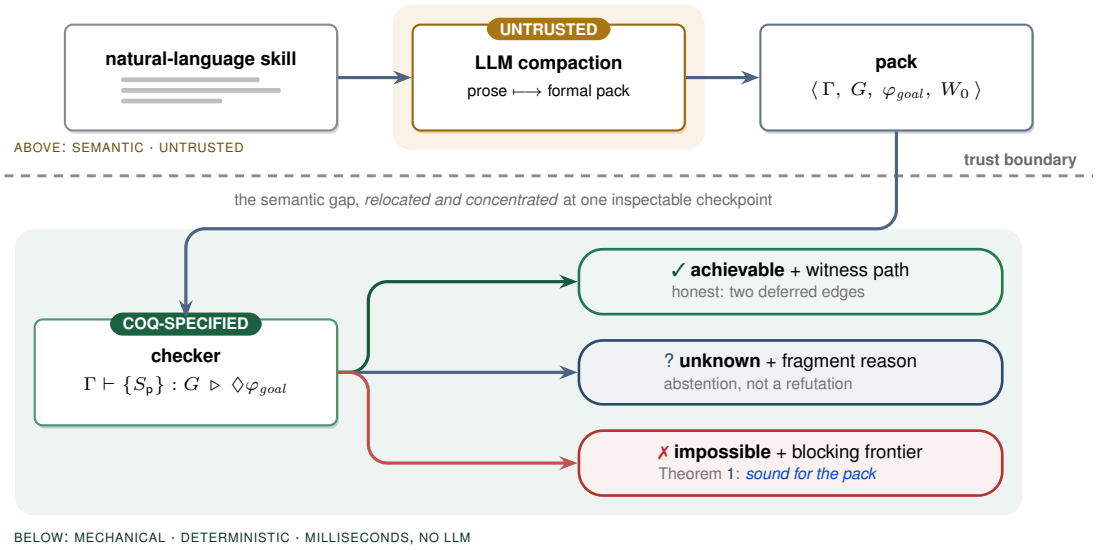


Figure 1: The trust boundary. An untrusted LLM compacts natural-language instructions into a formal pack. A deterministic checker then decides achievability over the pack using a Coq-verified abstraction theorem. The verdicts are asymmetric: *impossible* is sound (Theorem 1), while *achievable* depends on the accuracy of the pack (Section 3.1).

approach of SMPS [1, 2], it adds capabilities, explicit goals, and a refutation-sound but achievement-incomplete analysis for LLM-drafted skills.

2. **A mechanized soundness specification** (Appendix D) formalized in Coq. We prove a refutation-sound abstraction theorem parameterized by simulation hypotheses, together with tolerance soundness and capability monotonicity: adding capabilities cannot make an achievable goal impossible. We also establish operational correspondence through subject reduction and session fidelity for the direct judgment over a labelled transition system. For single-label communication, we mechanize the cases where other participants continue to act concurrently; the world-changing action cases are proved on paper under explicit side conditions.
3. **A termination boundary** (Appendix D). When the set of participants is fixed, the symbolic search always terminates, and an *impossible* verdict is sound. Exact achievability over unbounded integers is undecidable, so the checker works on a widened abstraction rather than deciding the exact judgment. Likewise, allowing an unbounded number of new participants to be added during execution can make the problem undecidable, because the search is no longer guaranteed to remain finite.
4. **An implementation and evaluation** (Sections 4–5). We implement the checker together with an LLM-compaction front-end and a schema gate. We evaluate skillc on every real use case we have run, from pre-flight checks and agent hosts to 447 real skills labelled by blind execution, with a named model and baselines, and we report where it falls short (Figures 3–4).

2 Overview by Example

Figure 2 follows one real skill through skillc, from its text to a *pack* to a verdict; the flight example below is the same idea in miniature.

Consider a one-agent skill whose goal is *a flight is booked and a confirmation is sent*. The LLM compacts the skill into a formal pack. If no email tool is available, there is no action that can establish confirmation_sent.

$$\varphi_{goal} \;=\; \mathsf{booked} \wedge \mathsf{confirmation_sent}$$
$$\Gamma(a) \;=\; \langle\, pre_a,\, \langle\, Add_a,\, Del_a,\, Asg_a,\, Nd_a \,\rangle \rangle$$

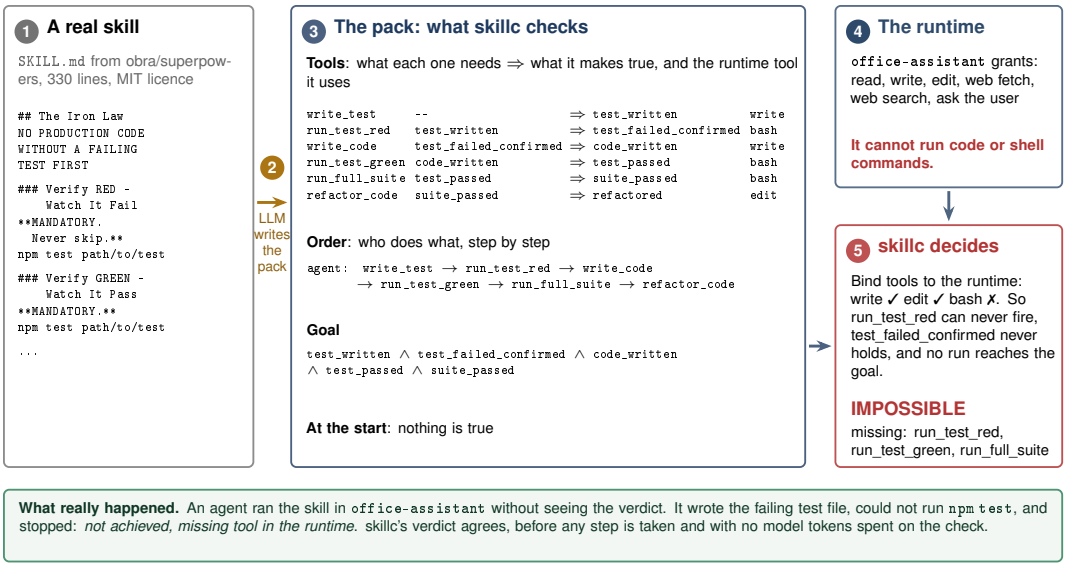


Figure 2: From a real skill to a verdict (batch 4 of Section 5). (1) The skill is prose. (2) An LLM turns it into a pack. (3) The pack lists the tools, each with what it needs and what it makes true, the order of the steps and who does them, the goal, and what is true at the start. (4) The runtime says which tools exist. (5) skillc binds the pack's tools to the runtime and searches for a run that reaches the goal; here none exists, and the verdict names the missing tools. The pack and the verdict are the stored ones from that run, re-checked with the current checker.

$$\begin{array}{lll} \mathsf{search} \mapsto \langle\, \top, & \langle\, \{\mathsf{searched}\}, & \emptyset,\, \emptyset,\, \emptyset \,\rangle\,\rangle \\ \mathsf{filter} \mapsto \langle\, \mathsf{searched}, & \langle\, \{\mathsf{filtered}\}, & \emptyset,\, \emptyset,\, \emptyset \,\rangle\,\rangle \\ \mathsf{book} \mapsto \langle\, \mathsf{filtered}, & \langle\, \{\mathsf{booked}\}, & \emptyset,\, \emptyset,\, \emptyset \,\rangle\,\rangle \\ \mathsf{email} \mapsto \langle\, \mathsf{booked}, & \langle\, \{\mathsf{confirmation_sent}\}, & \emptyset,\, \emptyset,\, \emptyset \,\rangle\,\rangle \end{array}$$

$$\mathrm{dom}(\Gamma_A) = \{\mathsf{search}, \mathsf{filter}, \mathsf{book}\} \qquad \mathrm{dom}(\Gamma_B) = \mathrm{dom}(\Gamma_A) \cup \{\mathsf{email}\}$$

Under STRIPS [17] frame semantics, the checker therefore returns *impossible* and identifies the missing capability. If the email tool is available, the goal is reachable through the sequence search · filter · book · email, which the checker returns as a witness. The same example is mechanized in Appendix D (the instance `FlightInstance`). Importantly, the booking itself remains reachable in the first case; the refutation concerns only the missing capability needed to complete the goal.

Tolerance is also important. A skill should not be rejected simply because an agent sends extra status messages, takes a clarifying step, or uses a different payload. Our analysis therefore allows these variations in three ways: *existential may-reachability*, where one successful path is sufficient; *session subtyping*, which allows safe interface variation; and *goal-relevant abstraction*, which ignores payload details that do not affect the goal. These mechanisms are defined in Appendices A–C.

3 Verification Model and Signals

Declared pack. The checker consumes $P = \langle \Gamma, G, \varphi_{goal}, W_0 \rangle$. Here, Γ contains capability declarations with guards and STRIPS-style effects; G is a goal-marked global protocol (a *global type*; Appendix A); φ_{goal} is the logical goal; and W_0 is the initial abstract world state, described by a constraint $init \in \mathcal{L}$. Any world satisfying this constraint is a possible initial world, written as $W_0 \vDash init$. The LLM produces P from the natural-language skill. A deterministic schema gate checks that the pack stays within the part of the language the checker can decide: a finite, fixed set of roles, guarded recursion, finite-domain booleans, and quantifier-free linear integer arithmetic. The formal syntax and semantics are given in Appendices A–B.

Verification signals. The checker uses three static signals:

Signal	Role	Output
tool availability and effects	capability reachability	missing capability or witness action
coordination structure	global protocol conformance	protocol mismatch or conformant protocol
goal and cost refinements	logical reachability	blocked guard or model

These signals are checked before execution. Runtime payload checks remain separate obligations and are not assumed by the static checker.

Decision procedure. After the schema gate, the checker (i) rejects undeclared capabilities and goal conditions that cannot be achieved by any available capability; (ii) checks role behaviors against G, requiring exact sender choices while allowing safe receiver-side branch supersets; and (iii) explores existential protocol/world successors from W_0, using Z3 [18] for guards and refinements. A goal condition that cannot be satisfied blocks the search. If the solver times out or the pack falls outside these restrictions, the checker returns UNKNOWN rather than *impossible*. The checker therefore returns:

$$\begin{array}{ll} \textsc{Impossible}(F) & \text{a sound explanation of why the goal cannot be reached,} \\ \textsc{Achievable}(\pi, O) & \text{a witness path } \pi \text{ and deferred obligations } O, \\ \textsc{Unknown}(r) & \text{an abstention with a reason } r. \end{array}$$

Each verdict records the pack digest, checker semantics, and artifact version.

Guarantees. Let a configuration contain both a protocol state and an abstract world state, and let abs map concrete configurations to abstract ones. Under the step and goal simulation assumptions for the declared pack (Appendix D), if the abstract search cannot reach φ_{goal}, then no concrete execution represented by that pack can reach the concrete goal (Theorem 1). This guarantee remains valid when the abstraction contains less detail, and adding more capabilities cannot cause an otherwise achievable goal to be reported as *impossible* (Theorems 2–3).

The direct conformance check also satisfies subject reduction and session fidelity (Theorems 4–5). The Coq development covers single-label communication together with concurrent actions by other participants. When two actions both change the world state, their interleaving requires an explicit commutativity assumption. The restrictions above ensure that the search terminates. An *impossible* verdict is sound for the declared pack, while an *achievable* verdict only states that the goal is reachable in the abstract search (Proposition 1).Allowing an unbounded number of new participants to be added during execution is an undecidable extension, so the implementation returns UNKNOWN in such cases. Full rules, assumptions, proofs, and mechanization scope appear in Appendix D.

3.1 Structured feedback and human oversight

The checker returns an explanation for IMPOSSIBLE and a witness path for ACHIEVABLE. These outputs provide concrete feedback rather than a single score. A bounded repair adapter may ask the LLM to fix a protocol, but it cannot invent capabilities or weaken the goal. Every revision is checked again by the deterministic checker. Human review remains necessary for the gap between the original intent and the formal pack. Automated prompt or scaffold search is an application of these verdicts, not an empirical contribution of this paper.

4 Implementation

We implemented the checker and its supporting components in Python. The compaction front-end sends the skill to an LLM and passes the result through a deterministic *schema gate* before the checker processes it. Packs that do not satisfy the schema are rejected at this stage. The checker implements the abstractions from Appendices A–C: STRIPS frame semantics for the world, existential search over the protocol, and Z3 for guard and goal satisfiability and arithmetic refinements. On success it returns a witness path; on refutation, it reports the reason: a missing capability, a guard that can never hold, or a goal condition that nothing can establish. A verdict has two separate parts. The *goal* part is ACHIEVABLE, IMPOSSIBLE or UNKNOWN. The *protocol* part is WELL_TYPED, REJECTED (reason NON_PROJECTABLE, NON_CONFORMANT, or MISSING_CAPABILITY for a tool missing only on a path the goal does not need) or NOT_CHECKED. A protocol rejection never becomes

IMPOSSIBLE. The reason is that T-COMM asks *every* branch of a choice to conform, so one bad branch rejects the protocol even when another branch reaches the goal; no theorem says that such a goal is unreachable. Each verdict also names the formal result that covers it, and a host lets a plan run only when the goal is reachable *and* the protocol is not rejected. The optional LLM front-end may use a NON_PROJECTABLE counterexample for one bounded repair round. It may not invent capabilities or weaken the goal, and the deterministic checker remains the final decision maker.

The conformance step ($G \vdash (\mathbb{M}; W)$, Section C.2) is implemented by means of a simple *type inference algorithm* since all typing rules are structural in $(\mathbb{M}; W)$. At each step we freely choose either one or two participant processes to reduce or a goal satisfied by the world obtaining the set of sessions which must be typed as premises of the typing rule.

The adapter also checks the participant side conditions of T-COMM/T-ACT/T-GOAL: it computes $\mathsf{prt}(G)$ and compares it with the roles for which the pack declares behaviour. A declared role outside $\mathsf{prt}(G)$ has contract End, so a non-trivial behavior for that role cannot conform. If the pack declares no behavior for a participant, the checker reports the missing role instead of leaving the assumption implicit.

5 Evaluation

We evaluate skillc on every real use case we have run (Figure 3; Appendix E lists each one with its source, size and labels). A rate is written as a percentage over the number of tests, for example 100% over 123 checks, and drawn with a Wilson 95% interval. Figure 4 gives the confusion matrices where every case has a label. Batches 1–4 used Claude sub-agents for every model step; other model runs use gpt-5.4.

Q1: real use cases. *A. No LLM in the loop.* The pack is read from the environment or written by hand. In a real workspace, seven workflows (from release reports to Word and Excel export), each run three times with the prerequisites probed automatically: skillc gave the correct verdict in 100% over 21 runs, stopped 100% over 12 blocked runs before any task step, used no model tokens and took a median of 32 ms; two gpt-5.4 agents stopped early in only 58% and 75% over the 12. In ten published Microsoft Agent 365 workflows, 100% over 123 checks (63 static, 60 runtime) gave the expected result, and 0% over 50 denied calls reached a tool.

B. The LLM writes the pack; the truth comes from running. Four disjoint batches of public skills, 447 skill–runtime pairs in sandbox runtimes, were run by an agent that did not see skillc's verdict; a pair is blocked when the run met a missing tool, account, network access or safety rule. skillc was right in 88% over 447 pairs, caught 84% over 145 blocked pairs, and refuted 10% over 302 working pairs (Figure 4c).

C. The LLM writes the pack; the truth comes from AI annotators. Claude sub-agents, blind to skillc, labelled two samples of 100 public skills for one agent session's runtime by checking each skill's requirements, with read-only probes (sample 1) or against the runtime's description (sample 2); no skill was run. There, what blocks a skill is rarely written in it (a refused host, a GPU, an account), and skillc catches only 23% over 31 and 39% over 18 blocked skills.

D. A running agent. In a hosted Agent Framework agent whose prompt no longer states the approval policy, the bare agent sent an unapproved report in 10% over 21 runs and the gated agent in 0% over 21. A guard that also reads the agent's reasoning caught 27% over 37 hazards in blind incident traces and flagged 2% over 97 benign steps.

Q2: against a direct LLM verdict. Figure 5 compares skillc with asking the same model directly, on the same cases, by accuracy, tokens and time. When the pack is read from the environment, skillc is more accurate (100% against 95% over 21 runs), uses no tokens against 1,574 per decision, and decides in 32 ms against 15 s. When an LLM must first write the pack (batches 1 and 4, 168 pairs), the two are equally accurate (89% and 90%), but skillc spends more tokens: 8,820 per pair to write the pack against 3,471 for the direct answer; the check itself then takes a median of 8 ms (90th percentile 17 ms). On samples 1 and 2 the direct verdict is more accurate (77% and 74% against 60% and 64%; tokens and time were not recorded there), but it refutes as many or more working skills (Figure 6). What skillc's extra tokens buy is the form of the answer: a refutation names its reason and blocking

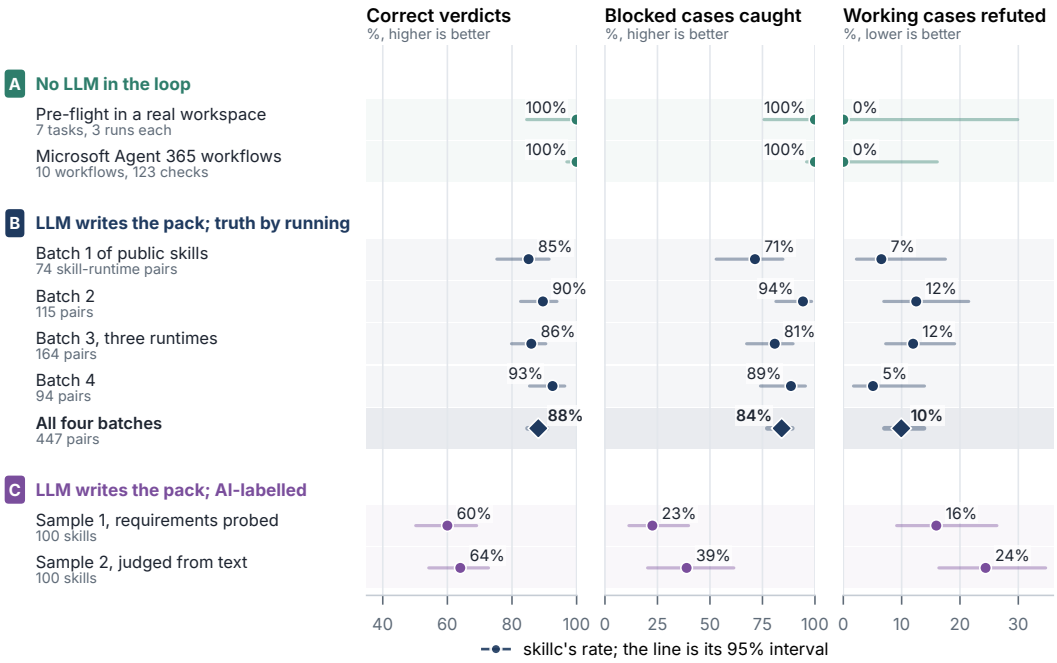


Figure 3: skillc on every real use case. Each row is one evaluation; the grey line under its name gives its size. A: the pack is read from the environment or written by hand, with no LLM. B: an LLM (a Claude model) writes the pack from a public skill, and the truth comes from running the skill in a sandbox without showing skillc's verdict; batches 1–4 are four disjoint samples of public skills. C: the same, but the truth comes from AI annotators (Claude sub-agents, blind to skillc) who checked each skill's requirements against the runtime without running it: with read-only probes in the live session (sample 1), or from the runtime's description alone (sample 2). Columns: the share of cases whose verdict matches the truth (a case with no verdict counts as not right); the share of cases that cannot run which skillc refutes; and the share of cases that can run which skillc wrongly refutes. Dots are rates, lines are 95% intervals.

frontier, an achievable verdict carries a witness path, the same pack always gives the same verdict, and the pack is what the runtime monitor enforces. A direct verdict is an answer only.

Q3: what the check costs. The checker itself uses no model tokens; only compaction does, once per skill version. The submitted version reported an $87\times$ saving from a cost model. Measured on 31 blocked files (Figure 7), an ungated gpt-5.4 agent used a median of 9,486 tokens before it stopped, against 6,289 for compaction and check: a saving of $1.58\times$ (95% CI 1.01–2.11). On working files the check costs about a third of one run. The case for the check is outcome, not cost: on 32% over 31 blocked files the ungated agent reported success on a task that could not be done.

6 Related Work

Multiparty session types [13, 16, 9] provide formal methods for checking communication protocols, including deadlock-freedom through projection and subtyping [14, 10]. Our protocol follows the synchronous formulation of this line of work [9–11]. We also follow the recent approach of checking a whole session configuration directly against a global type [1, 2], rather than first projecting the global type to local types. The new contribution is the world and goal layer: capability guards, effects, goal observation, and the asymmetric refutation guarantee are not modelled by classical MPST or new SMPS. These are not part of classical MPST [13, 16, 9] or the recent direct-checking formulation [1, 2].

Automated planning [17] decides whether actions with preconditions and effects can reach a goal; we apply the same idea to agent skills and combine it with multiparty protocols.

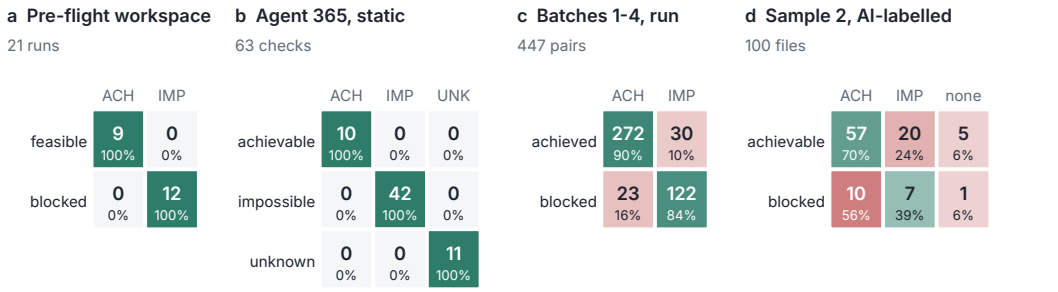


Figure 4: Confusion matrices for the use cases where every case has a label. (c) Batches 1–4 of category B, pooled. (d) Sample 2 of category C; "none" means no verdict: an abstention or a pack that did not pass the schema gate.

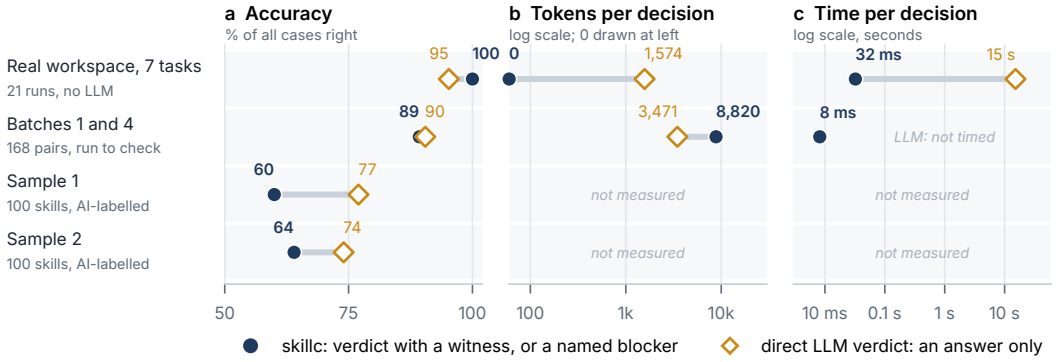


Figure 5: skillc against the simplest alternative, asking the same model directly whether the task can be done, on the same cases (Claude in batches 1 and 4, otherwise gpt-5.4). (a) Share of all cases whose verdict matches the truth; an abstention or a pack that fails the schema gate counts as not right. (b) Model tokens per decision: in the workspace, provider-reported; for batches 1 and 4, counted with a GPT-2 tokenizer, and skillc's tokens are those spent writing the pack, since the check itself uses none. (c) Time per decision: workspace medians include network and orchestration; for batches 1 and 4, the check alone, timed on the 120 packs of batch 4.

Agent-verification work has different goals: *message sequence charts* [20] check coordination, *VeriGuard* [21] verifies policies offline and monitors online, and *agent behavioural contracts* [22] enforce probabilistic compliance at runtime; none decides goal achievability before execution.

Skill-bundle scanners, NVIDIA's SkillSpector [23] and Verified Agent Skills [24], check a skill's code, metadata and dependencies for security and consistency before deployment. They are complementary: they inspect the original files, while we decide whether the declared goal can be reached.

Gating and filtering tools before an LLM chooses. Concurrently with and independently of this work, Ashkenazi et al. [31] deploy a deterministic gate in a customer-care assistant: for each skill, a hand-written predicate copies the skill's own abort conditions, and the gate hides the skill when the predicate holds on the live account state (59.4% of 1.75M skill–message pairs removed). It is one predicate per skill, checked once, with an informal soundness argument and no type system; it does not reason about steps, ordering, goals or handoffs. A group of preprints by Babu et al. models tools as sets of required and produced state variables, filters the menu to the next tool on a shortest path to a goal [26], learns such contracts with an LLM [27], asks a clarifying question when the goal is ambiguous [29], and benchmarks these filters [28]. Their state only grows (no delete effects), so reachability is easy and needs no proof, whereas with delete effects plan existence is PSPACE-complete [30]; they have no numbers, nondeterminism, branching, loops or roles, no impossibility verdict (a missing path counts as a filter failure, and their benchmark has no impossible task), and no formal guarantee. *Agentic model checking* [25] shares our design principle, "agents

propose, solvers verify": an LLM infers pre/postconditions for C and Rust functions and a bounded model checker checks them. It verifies code, not agent skills, and its results hold up to a loop bound, while our refutation holds for runs of any length and is proved in Coq.

To our knowledge, the work above does not combine capability-based goal reachability with multiparty protocol checking for agent skills, nor provide the same refutation-sound guarantee for an *impossible* verdict.

7 Limitations and Future Work

- **Dependence on the declared pack.** The guarantees apply to the declared Γ and S. If the LLM incorrectly describes a capability or omits an important detail from the skill, the checker can only reason about the resulting pack. A practical system should therefore also inspect `SKILL.md`, scripts, dependencies, manifests, and permission metadata before running the achievability check.
- **Adding participants during execution.** The current procedure terminates when the set of participants is fixed. When new participants can be added during execution, the procedure may return UNKNOWN rather than make an unsupported decision (Proposition 2).
- **Abstraction.** A coarse α allows more behaviors to be treated as equivalent. This preserves the refutation guarantee of Theorem 1, but may make the checker more likely to return *achievable*.
- **Formalization gaps.** The Coq development covers the generic refutation-sound abstraction theorem and concrete examples, head-move action safety, and subject reduction/session fidelity with full bystander interleaving for the single-label communication model, all axiom-free. The executable checker is schema-gated and tested against the formal specification, but its exact symbolic state is mechanized only as a *shallow embedding* (Lemma 3: constraints as predicates on numeric worlds; that the checker's QF-LIA formulas denote those predicates and that the SMT solver decides them correctly is trusted, not proved). The Coq model also does not yet combine participant matching, multi-branch choice, and world-changing interleaving in one development. Future work includes completing this mechanization, connecting interleaved session runs to the reachability search, proving equivalence between the declarative judgment and the projection-based adapter, and proving that abstract witnesses correspond to concrete sessions. Appendix D.3 lists exactly which step from a checker verdict to Theorem 1 is machine-checked, which is argued on paper, and which is trusted, and one over-claim of the checker that this revision found and fixed.
- **Recall on real files.** With LLM compaction, the checker misses most files that cannot run (39% caught over 18 on the blind set), because the model rarely writes down a needed host, GPU or account. Multi-actor files compact into one role unless the prompt names the roles, and without the environment's own state names the Agent 365 compaction gives no verdict at all (0 of 195 checks).
- **Evaluation scale.** The held-out sets have 100 files each and are now used up, so any further change to the front-end needs a new blind set; the Agent 365 gate results rest on hand-written packs.

Broader impact. Pre-execution checking can reduce wasted computation (Appendix F) and prevent structurally impossible plans from reaching execution. However, an incorrect pack can create false confidence, for example by omitting a tool effect or weakening the user's actual goal. Human review and runtime checks are still needed for consequential use. This work does not evaluate fairness, privacy, or deployment safety.

8 Conclusion

We decide before execution whether a skill's declared goal can be reached at all, with a deterministic checker over a pack derived from the skill. Its *impossible* verdict is proved sound for the pack, down to the checker's own search. In practice it is a cheap, explained pre-flight check and a gate that a running agent cannot argue past; its weak point is recall when an LLM writes the pack.

References

[1] Franco Barbanera, Mariangiola Dezani-Ciancaglini & Ugo de'Liguoro (2024): *Un-projectable Global Types for Multiparty Sessions*. In Alessandro Bruni, Alberto Momigliano, Matteo Pradella & Matteo Rossi, editors: *PPDP*, ACM Press, pp. 15:1–15:13,

[2] Francesco Dagnino, Paola Giannini & Mariangiola Dezani-Ciancaglini (2023): *Deconfined Global Types for Asynchronous Sessions*. *Logical Methods in Computer Science* 19(1), pp. 1–41,

[3] J. Li, X. Xiao, Y. Zhang, C. Liu, L. Zhao, X. Liao, Y. Ji, J. Wang, J. Gu, Y. Ge, W. Xu, X. Fang, X. Xu, T. Zhao, Y. Kim, T. Wang, J. Hamm, S. Krishnaswamy, J. Huan, C. Reddy. Agent Harness Engineering: A Survey. 2026.

[4] M. Chen, J. Wang, Z. Liu, Y. Wang, H. Zheng, Q. Wang. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. arXiv:2606.06324, 2026.

[5] Agent Skills Standard. Specification. https://github.com/agentskills/agentskills/blob/main/docs/specification.mdx, 2025.

[6] S. Hu, C. Lu, J. Clune. Automated Design of Agentic Systems. *ICLR*, 2025.

[7] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. AFlow: Automating Agentic Workflow Generation. *ICLR*, 2025.

[8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, I. Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025.

[9] N. Yoshida, L. Gheri. A Very Gentle Introduction to Multiparty Session Types. *ICDCIT*, LNCS 11969, 2020.

[10] S. Ghilezan, S. Jakšić, J. Pantović, A. Scalas, N. Yoshida. Precise Subtyping for Synchronous Multiparty Sessions. *J. Log. Algebr. Meth. Program.* 104, 2019.

[11] A. Scalas, N. Yoshida. Less is More: Multiparty Session Types Revisited. *Proc. ACM Program. Lang.* 3(POPL), 2019.

[12] K. Honda, V. T. Vasconcelos, M. Kubo. Language Primitives and Type Discipline for Structured Communication-Based Programming. *ESOP*, LNCS 1381, pp. 122–138, 1998.

[13] K. Honda, N. Yoshida, M. Carbone. Multiparty Asynchronous Session Types. *J. ACM* 63(1), 2016.

[14] S. Gay, M. Hole. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42(2–3), 2005.

[15] D. Brand, P. Zafiropulo. On Communicating Finite-State Machines. *J. ACM* 30(2), 1983.

[16] E. Li, F. Stutz, T. Wies, D. Zufferey. Complete Multiparty Session Type Projection with Automata. *CAV* 2023.

[17] R. Fikes, N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artif. Intell.* 2(3–4), 1971.

[18] L. de Moura, N. Bjørner. Z3: An Efficient SMT Solver. *TACAS*, LNCS 4963, pp. 337–340, Springer, 2008.

[19] C. Barrett, P. Fontaine, C. Tinelli. The SMT-LIB Standard: Version 2.6. Technical Report, Department of Computer Science, The University of Iowa, 2017. https://smt-lib.org.

[20] B. Bollig, M. Függer, T. Nowak. Provable Coordination for LLM Agents via Message Sequence Charts. *ISoLA*, 2026. arXiv:2604.17612.

[21] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, L. T. Le. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. arXiv:2510.05156, 2025.

[22] V. P. Bhardwaj. Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents. arXiv:2602.22302, 2026.

[23] N. Paz, K. Pradeep, N. Raghavan, A. Nikirk, M. Gupta. SkillSpector: A Pre-Publication Security Control for Agent Skills. *Agent Skills Workshop at CAIS*, 2026.

[24] NVIDIA. NVIDIA-Verified Agent Skills Provide Capability Governance for AI Agents. NVIDIA Technical Blog, 19 May 2026.

[25] Y. Sun, J. Liu, D. Kroening, J. Xue. Agentic Model Checking. arXiv:2605.21434, 2026.

[26] R. S. Babu, L. G. Iyer. ToolChoiceConfusion: Causal Minimal Tool Filtering for Reliable LLM Agents. arXiv:2606.06284, 2026.

[27] R. S. Babu, L. G. Iyer. Contract2Tool: Learning Preconditions and Effects for Reliable Tool-Augmented LLM Agents. arXiv:2606.07904, 2026.

[28] R. S. Babu, L. G. Iyer. ToolMenuBench: Benchmarking Tool-Menu Filtering Strategies for Reliable and Efficient LLM Agents. arXiv:2606.15508, 2026.

[29] R. S. Babu, R. Shukla. GIST-CMTF: Goal-State Inference for Causal Minimal Tool Filtering in LLM Agents. arXiv:2606.16813, 2026.

[30] T. Bylander. The Computational Complexity of Propositional STRIPS Planning. *Artif. Intell.* 69(1–2):165–204, 1994.

[31] O. Ashkenazi, V. Kloz, M. Ulianchenko. Don't Offer What Can't Be Done: Deterministic Executability Gating for LLM Skill Selection at Scale. arXiv:2608.01050, 2026.

A Syntax

We use disjoint sets of predicates $p \in \mathsf{Pred}$ (boolean), numeric variables $x \in \mathsf{Var}$, roles $\mathsf{p}, \mathsf{q}, \mathsf{r} \in \mathcal{R}$, capability names $a \in \mathsf{Cap}$, and message labels $\ell \in \mathsf{Lab}$. Following the modern presentation of multiparty sessions [1, 2] we define the model from the bottom up: state and capabilities (Sections A.1–A.2), then *processes* (Section A.3), and finally the global *type* they are checked against directly (Section A.4).

In each grammar, "::=" lists the possible forms of an expression. In each inference rule below, the statement below the line holds when all premises above the line hold. Rule names identify the corresponding checker obligation; they are not additional assumptions.

A.1 State logic

$$\begin{array}{rcll} e & ::= & x \mid n \mid e + e \mid e - e \mid c \cdot e & (c, n \in \mathbb{Z}) \\ \varphi & ::= & p \mid \top \mid \bot \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid e \bowtie e & \bowtie \in \{<, \leq, =, \geq, >, \neq\} \end{array}$$

The state logic $\mathcal{L}$ is quantifier-free linear integer arithmetic [19] with uninterpreted boolean predicates — a decidable fragment for which entailment $\vDash$ and satisfiability are discharged by a satisfiability-modulo-theories (SMT) solver.

A.2 Worlds and capabilities

A *world* $W = \langle B, N \rangle$ assigns truth values to predicates $B : \mathsf{Pred} \to \mathbb{B}$ and numeric variables $N : \mathsf{Var} \to \mathbb{Z}$. A *capability context* Γ maps each available tool a to a guarded effect:

$$\begin{aligned} \Gamma(a) &= \langle \mathit{pre}_a, \mathit{eff}_a \rangle, \qquad \mathit{pre}_a \in \mathcal{L}, \\ \mathit{eff}_a &= \langle \mathit{Add}_a \subseteq \mathsf{Pred},\ \mathit{Del}_a \subseteq \mathsf{Pred},\ \mathit{Asg}_a : \mathsf{Var} \rightharpoonup e,\ \mathit{Nd}_a : \mathsf{Var} \rightharpoonup \mathcal{L} \rangle \end{aligned}$$

$$a \notin \mathrm{dom}(\Gamma) \iff \text{the agent does } \textit{not} \text{ possess tool } a \quad \text{(no hallucinated tools)}.$$

$\mathit{Add}/\mathit{Del}$ are the STRIPS add- and delete-lists (well-formedness requires $\mathit{Add}_a \cap \mathit{Del}_a = \emptyset$, so the order of union and difference in apply below is immaterial); Asg gives deterministic numeric updates $x := e$; Nd gives *nondeterministic* updates $x := *$ constrained by a formula over the new value (used to model post-conditions such as *"books a fare under 500"*).

A.3 Processes and multiparty sessions

Skills *execute* as processes; a multi-agent run is a session of named processes. We use the synchronous multiparty session calculus in its modern coinductive presentation [1, 2]: processes coinductively defined are possibly infinite *regular* trees (finitely many distinct subtrees), and labels are goal-relevant

tokens, $\ell = \alpha(m)$ for concrete messages m — the tolerance dial α is a parameter of the label alphabet.

$$P \ ::=^{coind} \ \mathbf{0} \mid \mathsf{p}!\{\ell_i.\, P_i\}_{i\in I} \mid \mathsf{p}?\{\ell_i.\, P_i\}_{i\in I} \mid a.\, P \qquad\qquad \mathbb{M} \ = \ \mathsf{p}_1[P_1] \parallel \cdots \parallel \mathsf{p}_n[P_n]$$

with I finite and non-empty, labels ℓ_i pairwise distinct, and role names pairwise distinct. The output $\mathsf{p}!\{\ell_i.P_i\}$ *internally* chooses a label and sends it to p; the input $\mathsf{p}?\{\ell_i.P_i\}$ offers an *external* choice; $a.P$ invokes capability a — the only construct that changes the world. A skill S_p *is* a process: the behaviour declared for role p.

A.4 Global types

The single communication construct of a global type is a *directed* labelled choice — selection by p and branching at q in one construct. (A partnerless "p selects" admits no coherent view for the roles that did not choose; directed choice is the standard MPST construct [13, 9].)

$$G \ ::=^{coind} \ \mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i\in I} \mid a@\mathsf{p}.\, G \mid \checkmark\varphi.\, G \mid \mathsf{End}$$

subject to $\mathsf{p} \neq \mathsf{q}$ and the same regularity and label conditions as processes. This is the *only* type in the discipline: Section C.2 types a whole session directly against G — there is no local-type grammar, no projection function, and no separate subtyping relation. Goal markers $\checkmark\varphi$ live *only* in G: achievability is decided on the global type (Rule ACH in C.3), and markers are consumed at typing time by T-GOAL without ever burdening a process. Only $a@\mathsf{p}$ changes the world; messages carry coordination information only.

B Operational Semantics

B.1 World transitions

A capability fires when its guard holds, producing a successor world. Because Nd is nondeterministic, $\langle W\rangle\, a\, \langle W'\rangle$ may relate one world to many. The relation is implicitly indexed by Γ: the notation pre_a, eff_a is defined only when $\Gamma(a) = \langle pre_a, \mathit{eff}_a\rangle$, so every firing presupposes $a \in \mathrm{dom}(\Gamma)$.

$$\textsc{World-Act}\ \frac{W \vDash pre_a \qquad W' \in \mathrm{apply}(\mathit{eff}_a, W)}{\langle W\rangle\ a\ \langle W'\rangle}$$

$$\mathrm{apply}(\mathit{eff}_a, \langle B, N\rangle) = \langle B', N'\rangle, \qquad B' = (B \cup Add_a) \setminus Del_a$$

$$N'(x) = \begin{cases} [\![Asg_a(x)]\!]_N & \text{if } x \in \mathrm{dom}(Asg_a) \\ v \text{ with } \langle B, N[x{\mapsto}v]\rangle \vDash Nd_a(x) & \text{if } x \in \mathrm{dom}(Nd_a) \\ N(x) & \text{otherwise (frame)} \end{cases}$$

$[\![e]\!]_N$ is the usual integer evaluation of e, reading each variable from N. When several variables are in $\mathrm{dom}(Nd_a)$, each constraint is evaluated with only its own variable updated; the schema gate rejects an $Nd_a(x)$ that mentions another updated variable.

B.2 A labelled semantics for sessions and global types

Only capabilities touch the world, so sessions reduce as configurations $(\mathbb{M}; W)$; communication is synchronous [10]. Each transition carries a label Λ recording its *participants*, so that independent moves by disjoint roles can be identified. Two auxiliary maps drive the labelled system: the *participants* $\mathsf{prt}(\cdot)$ of a global type, a label, or a session, and the *capabilities* $\mathsf{cap}(\cdot)$ (the set of moves a global type can perform). On global types,

$$\mathsf{prt}(\mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i\in I}) = \{\mathsf{p},\mathsf{q}\} \cup \textstyle\bigcup_{i\in I} \mathsf{prt}(G_i) \qquad \mathsf{prt}(\checkmark\varphi.G) = \mathsf{prt}(G)$$
$$\mathsf{prt}(a@\mathsf{p}.G) = \{\mathsf{p}\} \cup \mathsf{prt}(G) \qquad\qquad \mathsf{prt}(\mathsf{End}) = \emptyset,$$

on labels $\mathsf{prt}(\mathsf{p}\,\ell\,\mathsf{q}) = \{\mathsf{p},\mathsf{q}\}$, $\mathsf{prt}(a@\mathsf{p}) = \{\mathsf{p}\}$, $\mathsf{prt}(\checkmark\varphi) = \emptyset$, and on sessions $\mathsf{prt}(\mathbb{M}) = \{\mathsf{p} \mid \mathbb{M} \equiv \mathsf{p}[P] \ \&\ P \neq \mathbf{0}\}$ (the roles with a residual behaviour). The capabilities of a global type are

$$\mathsf{cap}(\mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i\in I}) = \{\mathsf{p}\,\ell_i\,\mathsf{q} \mid i \in I\} \cup \textstyle\bigcup_{i\in I} \mathsf{cap}(G_i)$$
$$\mathsf{cap}(a@\mathsf{p}.G) = \{a@\mathsf{p}\} \cup \mathsf{cap}(G) \qquad \mathsf{cap}(\checkmark\varphi.G) = \{\checkmark\varphi\} \cup \mathsf{cap}(G) \qquad \mathsf{cap}(\mathsf{End}) = \emptyset.$$

The capability names used by a protocol are

$$\mathsf{caps}(G) = \{a \mid \exists \mathsf{p}.\ a@\mathsf{p} \in \mathsf{cap}(G)\}.$$

The LTS of sessions is defined by

S-COMM
$$\frac{k \in I \subseteq J}{(\mathsf{p}[\mathsf{q}!\{\ell_i.P_i\}_{i\in I}] \parallel \mathsf{q}[\mathsf{p}?\{\ell_j.Q_j\}_{j\in J}] \parallel \mathbb{M}; W) \xrightarrow{\mathsf{p}\,\ell_k\,\mathsf{q}} (\mathsf{p}[P_k] \parallel \mathsf{q}[Q_k] \parallel \mathbb{M}; W)}$$

S-ACT
$$\frac{\langle W\rangle\ a\ \langle W'\rangle}{(\mathsf{p}[a.P] \parallel \mathbb{M}; W) \xrightarrow{a@\mathsf{p}} (\mathsf{p}[P] \parallel \mathbb{M}; W')}$$

where $\parallel$ is commutative, associative and $\mathsf{p}[\mathbf{0}] \parallel \mathbb{M} \equiv \mathbb{M}$. A non-terminated configuration with no successor is *stuck*. The planner/worker deadlock from Section 1 is one example: both processes wait for an input, so neither can take the next step. A goal marker is not part of any process, and satisfying the goal does not create a session transition. Instead, the predicate $goal_{\varphi_{goal}}$ below checks whether the goal holds. Session labels Λ therefore contain only communications and actions, as in the Coq development. On the type side, the goal marker is discharged together with the next step by G-GOAL-F below.

The checker relates a session to its protocol through a *labelled* transition system on global types, $(G, W) \rightsquigarrow_\Lambda (G', W')$, built from *head* rules that consume the leading construct and *interleaving* rules that let a move by participants *disjoint* from the head commute inward:

G-COMM-E
$$\frac{k \in I}{(\mathsf{p}\to\mathsf{q}:\{\ell_i.G_i\}_{i\in I},\ W) \rightsquigarrow_{\mathsf{p}\,\ell_k\,\mathsf{q}} (G_k,\ W)}$$

G-ACT-E
$$\frac{\langle W\rangle\ a\ \langle W'\rangle}{(a@\mathsf{p}.\ G,\ W) \rightsquigarrow_{a@\mathsf{p}} (G,\ W')}$$

G-GOAL-E
$$\frac{W \vDash \varphi}{(\checkmark\varphi.\ G,\ W) \rightsquigarrow_{\checkmark\varphi} (G,\ W)}$$

G-COMM-I
$$\frac{\forall i \in I.\ \big((G_i, W) \rightsquigarrow_\Lambda (G_i', W')\ \wedge\ \Lambda \in \mathsf{cap}(G_i)\big) \qquad \mathsf{prt}(\Lambda) \cap \{\mathsf{p},\mathsf{q}\} = \emptyset}{(\mathsf{p}\to\mathsf{q}:\{\ell_i.G_i\}_{i\in I},\ W) \rightsquigarrow_\Lambda (\mathsf{p}\to\mathsf{q}:\{\ell_i.G_i'\}_{i\in I},\ W')}$$

G-ACT-I
$$\frac{(G, W) \rightsquigarrow_\Lambda (G', W') \qquad \Lambda \in \mathsf{cap}(G) \qquad \mathsf{prt}(\Lambda) \cap \{\mathsf{p}\} = \emptyset}{(a@\mathsf{p}.\ G,\ W) \rightsquigarrow_\Lambda (a@\mathsf{p}.\ G',\ W')}$$

G-GOAL-F
$$\frac{W \vDash \varphi \qquad (G, W) \rightsquigarrow_\Lambda (G', W')}{(\checkmark\varphi.\ G,\ W) \rightsquigarrow_\Lambda (G',\ W')}$$

In G-COMM-I, all branches take the same Λ and reach the same W'. For communication, $W' = W$, which is the only case needed here. G-GOAL-E discharges a goal marker when $W \vDash \varphi$. In the achievability search, an unsatisfied goal marker blocks that path. In the labelled correspondence, G-GOAL-F discharges the marker and takes the next step in the same transition, as in `GS_Goal` in the Coq development. The goal is checked in the world state where the step occurs, so no goal-stability assumption is needed. The goal marker $\checkmark\varphi$ is not a label in the correspondence.

The extraction (head) rules G-COMM-E/G-ACT-E/G-GOAL-E give the Λ-erased reduction $(G, W) \rightsquigarrow (G', W')$ that the achievability judgment below uses; the interleaving rules matter only for the operational correspondence of Section D.4, and their $\Lambda \in \mathsf{cap}(G)$ premise confines a commuting move to a capability the residual protocol actually offers. G-COMM-E is legitimate because choice is directed — the branch is a communication both endpoints observe, and Section C.2 checks that every third party can follow. A well-formed pack uses its declared goal φ_{goal} at each goal marker. A configuration is *goal-satisfying* when control is at such a marker or at the terminus and the world entails that declared goal:

$$goal_{\varphi_{goal}}(\checkmark\varphi_{goal}.G, W) \iff W \vDash \varphi_{goal}, \qquad goal_{\varphi_{goal}}(\mathsf{End}, W) \iff W \vDash \varphi_{goal},$$

$$\Gamma; G \vDash \Diamond_{init}\varphi_{goal} \iff \exists W_0 \vDash init.\ \exists (G', W').\ (G, W_0) \rightsquigarrow^* (G', W') \ \wedge\ goal_{\varphi_{goal}}(G', W').$$

The diamond is *existential*: a single witnessing run suffices — the formal source of detour-tolerance.

B.3 Realizability and subtyping, without projection

The classical route to ruling out a deadlocking handoff is *projection*: compute each role's local type $G\upharpoonright \mathsf{r}$ by structural recursion on G, taking the *merge* of a choice's branches at every role that neither sends nor receives, and declaring G *realizable* exactly when this partial function is defined everywhere — undefinedness of the merge is the static signature of a role forced to behave differently in branches it cannot distinguish. Session subtyping $\leq$ is then a second, separate coinductive relation on local types, letting a role's actual behaviour offer more external choices and fewer internal ones than its projected contract demands [14, 10, 11].

Both devices exist to answer questions about the *network as a whole* — "can every role tell the branches apart it needs to?", "may this implementation stand in for that contract?" — while type-checking only one role's syntax at a time. Section C.2 answers the same two questions with a single judgment, *typing the whole configuration directly against* G: a choice node's rule quantifies over every branch the protocol declares and requires the *same* residual configuration to check against each one. This is the conservative plain-merge condition, obtained without ever computing $\sqcap$ or a local type. The receiver-side subtyping direction (a receiver may accept a superset of the protocol's labels) is folded into the same rule as a label-set inclusion $I \subseteq J$. Tolerance therefore still has the same three independent knobs from Section 2 — the existential $\Diamond$ (Section B.2), subtyping (now inside T-COMM, Section C.2), and the granularity of α — "flexible, tolerant of small details" is still the profile $\langle$coarse α, $\Diamond$, linear $\mathcal{L}\rangle$; a later safety layer flips the same machinery to the universal $\Box$ over permission predicates.

C The Achievability Type System

Achievability now factors into three premises, each checked by a separate terminating procedure , combined by the top rule ACH: no hallucinated capability, direct conformance of the declared skills to G, and reachability of the goal.

C.1 Capability and goal typing

$$\text{T-EFF}\ \frac{\Gamma(a) = \langle pre, \mathit{eff}\rangle \qquad \varphi \vDash pre}{\Gamma \vdash \{\varphi\}\, a\, \{\mathrm{sp}(\varphi, \mathit{eff})\}} \qquad \text{T-MISS}\ \frac{a \notin \mathrm{dom}(\Gamma)}{\Gamma \vdash a \triangleright \text{✗}\ (\text{MISSING_CAPABILITY})}$$

$\mathrm{sp}(\varphi, \mathit{eff})$ is the strongest postcondition of the STRIPS effect. T-MISS fires on hallucinated planning: the plan names a tool the context does not grant, and achievability is refuted without any search.

C.2 Direct conformance typing

Skills are typed *directly* against the global protocol and the world, by the coinductive judgment $G \vdash (\mathbb{M}; W)$ — read "G types session $\mathbb{M}$ starting from world W." There is no local type, no projection, and no separate subtyping relation: the receiver-side subtyping direction and the unobserved-choice

check are structural side conditions of one rule.

$$\textsc{T-End}\quad \frac{}{\mathsf{End} \vdash (\mathsf{p}[\mathbf{0}]; W)}$$

$$\textsc{T-Act}\quad \frac{\exists W'.\ \langle W\rangle\, a\, \langle W'\rangle \qquad \forall W'.\ \langle W\rangle\, a\, \langle W'\rangle \implies G \vdash (\mathsf{p}[P] \parallel \mathbb{M}; W') \qquad \mathsf{prt}(G) \cup \{\mathsf{p}\} = \mathsf{prt}(\mathbb{M}) \cup \{\mathsf{p}\}}{a@\mathsf{p}.\, G \vdash (\mathsf{p}[a.P] \parallel \mathbb{M}; W)}$$

$$\textsc{T-Goal}\quad \frac{G \vdash (\mathbb{M}; W) \qquad W \vDash \varphi \qquad \mathsf{prt}(G) = \mathsf{prt}(\mathbb{M})}{\checkmark\varphi.\, G \vdash (\mathbb{M}; W)}$$

$$\textsc{T-Comm}\quad \frac{G_i \vdash (\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}; W) \qquad \forall i \in I \subseteq J \qquad \mathsf{prt}(\mathsf{p}{\to}\mathsf{q} : \{\ell_i.G_i\}_{i\in I}) = \mathsf{prt}(\mathbb{M}) \cup \{\mathsf{p},\mathsf{q}\}}{\mathsf{p}{\to}\mathsf{q} : \{\ell_i.G_i\}_{i\in I} \vdash (\mathsf{p}[\mathsf{q}!\{\ell_i.P_i\}_{i\in I}] \parallel \mathsf{q}[\mathsf{p}?\{\ell_j.Q_j\}_{j\in J}] \parallel \mathbb{M}; W)}$$

T-Act checks the protocol and world state together with World-Act. The type $a@\mathsf{p}.G$ is checked before the action in W, and the remaining type G is checked after the action in W' — the same convention as G-Act-E (Section B.2). The second premise considers every possible successor world because an Nd effect may produce different outcomes. Subject reduction therefore requires the remaining type to be valid in each successor. For capabilities without Nd, there is only one successor, so the rule is unchanged.

In T-Comm the sender p offers exactly the protocol's branch set I, while the receiver q may accept *more* ($I \subseteq J$) — the one subtyping direction the rule keeps (safe interface slack at the receiver). Requiring *every* protocol branch ℓ_i to be checked against the *same* bystanders $\mathbb{M}$ is the plain-merge form of the unobserved-choice condition, so a role forced to behave differently across branches it cannot distinguish makes no derivation possible — the rule fails closed, and deadlock-freedom falls out of T-Comm itself with no merge to compute (Theorem 4). Each rule additionally carries a *participant-matching* side condition tying the protocol's participants to the session's: $\mathsf{prt}(\mathsf{p}{\to}\mathsf{q} : \{\ell_i.G_i\}) = \mathsf{prt}(\mathbb{M}) \cup \{\mathsf{p},\mathsf{q}\}$ in T-Comm, $\mathsf{prt}(G) \cup \{\mathsf{p}\} = \mathsf{prt}(\mathbb{M}) \cup \{\mathsf{p}\}$ in T-Act, and $\mathsf{prt}(G) = \mathsf{prt}(\mathbb{M})$ in T-Goal. These make the invariant "the roles the protocol still mentions are exactly the roles still running" hold at every node, ruling out a session that carries a stray participant the protocol has forgotten (or vice versa) — the well-formedness that lets the interleaving cases of Section D.4 match a bystander move on the session to a $\mathsf{cap}(\cdot)$-labelled move on the type. T-End closes the discipline at the terminus: the terminated protocol End types a single idle role $\mathsf{p}[\mathbf{0}]$ — and, through the congruence $\mathsf{p}[\mathbf{0}] \parallel \mathbb{M} \equiv \mathbb{M}$, any session all of whose roles have terminated — the base case of the participant invariant, since $\mathsf{prt}(\mathsf{End}) = \emptyset$ and an idle role contributes nothing to $\mathsf{prt}(\mathbb{M})$. T-Act is derivable only when $\langle W\rangle\, a\, \langle W'\rangle$ holds, which itself presupposes $a \in \mathrm{dom}(\Gamma)$: this is where hallucinated capabilities die, with T-Miss as the diagnostic.

C.3 The achievability judgment

$$\textsc{Ach}\quad \frac{\mathsf{caps}(G) \subseteq \mathrm{dom}(\Gamma) \qquad \exists W_0 \vDash \mathit{init}.\ G \vdash (\mathsf{p}_1[S_{\mathsf{p}_1}] \parallel \cdots \parallel \mathsf{p}_n[S_{\mathsf{p}_n}]; W_0) \ \wedge\ \Gamma; (G, W_0) \vDash \Diamond\varphi_{goal}}{\Gamma \vdash \{S_{\mathsf{p}}\}_{\mathsf{p}} \ :\ G \triangleright \Diamond\varphi_{goal}}$$

where $\{\mathsf{p}_1, \ldots, \mathsf{p}_n\} = \mathsf{prt}(G)$: the judgment takes one declared skill S_{p} per role of the protocol. Read top-to-bottom: no hallucinated tools; some plausible initial world exists from which the declared skills directly conform to the protocol (deadlock-free, every capability call guarded) *and* that same world reaches the goal. The reachability half is unchanged from Section B.2 and decided on Γ, G alone, before any S_{p} is known — exactly what lets the checker run on the LLM-compacted pack the moment it exists; conformance is the additional, separately checked premise once the skills are declared, and needs no transport lemma: the receiver-side subtyping slack is already inside T-Comm.

C.4 Reachability rules

The diamond is derived by the following inductive system, realized by the checker as a finite forward search over $\rightsquigarrow$; unlike T-COMM/T-ACT/ T-GOAL, it never mentions a session $\mathbb{M}$, which is what makes it checkable on the pack alone.

$$\textsc{Reach-Goal}\quad \frac{W \vDash \varphi_{goal} \qquad G = \mathsf{End} \ \vee \ \exists G'.\, G = \checkmark\varphi_{goal}.G'}{\Gamma;(G,W) \vDash \Diamond\varphi_{goal}}$$

$$\textsc{Reach-Step}\quad \frac{(G,W) \rightsquigarrow (G',W') \qquad \Gamma;(G',W') \vDash \Diamond\varphi_{goal}}{\Gamma;(G,W) \vDash \Diamond\varphi_{goal}} \qquad \textsc{Reach-Or}\quad \frac{\exists k \in I.\, \Gamma;(G_k,W) \vDash \Diamond\varphi_{goal}}{\Gamma;(\mathsf{p}\to\mathsf{q}{:}\{\ell_i.G_i\}_{i\in I},W) \vDash \Diamond\varphi_{goal}}$$

REACH-OR is derivable from REACH-STEP with G-COMM-E; we keep it because the search implements branching this way.

D Metatheory

We separate what is *mechanized in Coq* (Theorems 1–3; Theorems 4–5 for the single-label communication fragment with full interleaving; the head-move action case; and the concrete instances, all axiom-free; and Lemma 3 for a shallow embedding of the state logic) from what is *proved on paper* (the action-interleaving cases, the refinement of that embedding to the checker's formulas and solver, termination, and the undecidability boundary).

D.1 Soundness of the abstraction

Let $\mathcal{C}$ be the concrete configuration space of protocol/world pairs, $\mathcal{A}$ the abstract configuration space the checker explores, and $abs \subseteq \mathcal{C} \times \mathcal{A}$ the abstraction *relation*: a concrete configuration may be represented by several abstract ones, because the accumulated constraint of Lemma 3 depends on the path the search took, not on the world alone. We write $abs(C,A)$ for "A represents C" and require:

(step-sim) $C \rightsquigarrow C' \wedge abs(C,A) \Rightarrow \exists A'.\, A \mathrel{\hat{\rightsquigarrow}} A' \wedge abs(C',A')$, **(goal-sim)** $goal_{\varphi_{goal}}(C) \wedge abs(C,A) \Rightarrow \hat{goal}(A)$.

When abs is the graph of a function the two conditions reduce to the familiar functional form, mechanized as the special case `refutation_sound_fun_is_rel`.

Lemma 1 (Transport)**.** *If $C_0 \rightsquigarrow^* C$ and $abs(C_0,A_0)$, then there is an A with $A_0 \mathrel{\hat{\rightsquigarrow}}^* A$ and $abs(C,A)$.*

Proof. By induction on the reflexive-transitive closure. The empty run is reflexivity with $A = A_0$. For $C_0 \rightsquigarrow^* C' \rightsquigarrow C$, the induction hypothesis gives an A' with $A_0 \mathrel{\hat{\rightsquigarrow}}^* A'$ and $abs(C',A')$, and STEP-SIM gives an A with $A' \mathrel{\hat{\rightsquigarrow}} A$ and $abs(C,A)$; compose them. Mechanized as `reach_abs_rel`, parameterized by STEP-SIM (`reach_abs` is the functional special case). □

Lemma 2 (Reachability monotonicity)**.** *If every step of $\rightsquigarrow_1$ is a step of $\rightsquigarrow_2$ (that is, $x \rightsquigarrow_1 y$ implies $x \rightsquigarrow_2 y$), then $s \rightsquigarrow_1^* w$ implies $s \rightsquigarrow_2^* w$.*

Proof. By induction on the closure $s \rightsquigarrow_1^* w$. The empty run transports by reflexivity. For $s \rightsquigarrow_1^* u \rightsquigarrow_1 w$, the induction hypothesis gives $s \rightsquigarrow_2^* u$; the hypothesis turns the last step $u \rightsquigarrow_1 w$ into $u \rightsquigarrow_2 w$; appending it gives $s \rightsquigarrow_2^* w$. Mechanized as `reach_mono`. □

D.2 Refutation soundness, tolerance, monotonicity

Theorem 1 (Refutation soundness)**.** *If the abstract system has no reachable goal state from an abstract configuration A_0 with $abs((G,W_0),A_0)$, then no declared run of that pack configuration reaches φ_{goal}. Hence an* impossible *verdict is never wrong relative to the pack, provided* STEP-SIM *and* GOAL-SIM *hold.*

Proof. We prove the contrapositive: if *some* declared run reaches the goal, then the abstract system reaches an abstract goal state. Suppose $C_0 \rightsquigarrow^* C$ with $goal_{\varphi_{goal}}(C)$. By Lemma 1, there is an A with $A_0 \hat{\rightsquigarrow}^* A$ and $abs(C, A)$, and by GOAL-SIM, $\hat{goal}(A)$. Thus A is an abstract goal state reachable from A_0, contradicting the hypothesis. Hence no declared run reaches φ_{goal}: a verdict of *impossible* — read off exactly the premise, that the abstract search exhausts without hitting an abstract goal — is never wrong. Mechanized, axiom-free as a theorem schema parameterized by the two simulation hypotheses (the existential witness is A; the relational form is `refutation_sound_rel`, and the functional form below is its special case `refutation_sound_fun_is_rel`): □

```
Theorem refutation_sound :  forall s0,
(~ exists a, reach astep (abs s0) a /\ agoal a) ->
(~ exists w, reach cstep s0 w /\ cgoal w).
(* Print Assumptions refutation_sound.  => Closed under the global
context *)
```

Lemma 3 (The symbolic abstraction satisfies the hypotheses)**.** *Let abs relate $(G, \langle B, N\rangle)$ to $(G, \langle B, \psi\rangle)$ for every accumulated constraint $\psi \in \mathcal{L}$ with $N \vDash \psi$. Let an abstract step apply the same Add/Del update to B and replace ψ by the strongest postcondition of the numeric effect, and let $\hat{goal}(G, \langle B, \psi\rangle)$ hold when goal holds at control G and $\psi \wedge \varphi_{goal}$ is satisfiable under B. Then* STEP-SIM *and* GOAL-SIM *hold, and the widening $\psi \mapsto \top$ preserves both.*

Proof. GOAL-SIM: if $\langle B, N\rangle \vDash \varphi_{goal}$ and $N \vDash \psi$, then N itself witnesses satisfiability of $\psi \wedge \varphi_{goal}$ under B. STEP-SIM: a concrete firing needs $\langle B, N\rangle \vDash pre_a$, so $\psi \wedge pre_a$ is satisfiable (N witnesses it) and the symbolic edge exists; the boolean update is identical on both sides, and the concrete successor satisfies the strongest postcondition by definition, so it is related to the abstract successor. Widening replaces ψ by $\top$ on the edges the implementation chooses; $N' \vDash \top$ trivially, so STEP-SIM holds for every widening policy, and by Theorem 2 widening can only enlarge the reachable set. Hence Theorem 1 applies to the search the implementation runs. Mechanized as `SymbolicInstance.symbolic_refutation_sound` for a shallow embedding of the state logic: constraints are predicates on numeric worlds, guards and effects are relations, the strongest postcondition is the image, the widening is an arbitrary policy on edges, and the abstraction is the relation $N \vDash \psi$. What remains on paper is the refinement of that embedding to the checker: that its QF-LIA formulas denote those predicates and that the SMT solver decides their satisfiability correctly (Section 7). □

Theorem 2 (Tolerance soundness)**.** *If a* coarser *over-approximation (more edges) cannot reach the goal, neither can any finer system. Hence coarsening α to tolerate more detail never produces a false refutation.*

Proof. Let $\rightsquigarrow_{\text{fine}}$ and $\rightsquigarrow_{\text{coarse}}$ be the two abstract step relations, with the coarsening an *over*-approximation: every fine step is also a coarse step, $x \rightsquigarrow_{\text{fine}} y \Rightarrow x \rightsquigarrow_{\text{coarse}} y$. Suppose, for contradiction, that the finer system reaches the goal: $s_0 \rightsquigarrow^*_{\text{fine}} a$ with $\hat{goal}(a)$. By Lemma 2 (with $\rightsquigarrow_1 = \rightsquigarrow_{\text{fine}}$, $\rightsquigarrow_2 = \rightsquigarrow_{\text{coarse}}$), $s_0 \rightsquigarrow^*_{\text{coarse}} a$; and $\hat{goal}(a)$ still holds. So the coarser system reaches a goal state, contradicting the hypothesis. Hence no coarser-refuted goal is reachable in any finer system: coarsening α only *adds* abstract edges, so it can never turn an unreachable goal into a reachable one — refutation is preserved. Mechanized as `tolerance_sound`, axiom-free. □

Theorem 3 (Capability monotonicity)**.** *If $\Gamma \subseteq \Gamma'$ and Γ reaches φ_{goal}, then Γ' reaches φ_{goal}. Granting tools never makes an achievable goal impossible.*

Proof. Enlarging the capability context only *adds* guarded effects: since $\Gamma \subseteq \Gamma'$, every capability firing available under Γ is available under Γ' with the same guard and effect (WORLD-ACT in B.1 quantifies over $a \in \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma')$), so every step of the Γ-transition relation is a step of the Γ'-relation. Suppose Γ reaches the goal configuration C from C_0. By Lemma 2, the same configuration is reachable under Γ', and $goal_{\varphi_{goal}}(C)$ is unchanged because the goal predicate does not depend on Γ. Hence Γ' reaches φ_{goal} via the same witnessing run. The other premises of ACH are monotone in the same direction ($\mathsf{caps}(G) \subseteq \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma')$ still holds, and realizability and conformance do not mention Γ beyond $a \in \text{dom}(\Gamma)$), so the whole judgment is preserved. The transport half is mechanized as `cap_monotone`, axiom-free, stated over an inclusion of step relations; the reduction of $\Gamma \subseteq \Gamma'$ to that inclusion is the WORLD-ACT argument above, on paper. □

Concrete instance. The flight skill of Section 2, variant A, is mechanized as `FlightInstance`. We prove `flight_refuted` (no run reaches booked ∧ confirmation_sent, since no step touches conf) and `booking_reachable` (a booking is still reachable). Both are axiom-free, witnessing non-vacuity of Theorem 1.

D.3 From a checker verdict to Theorem 1

The reviews asked which exact path leads from an IMPOSSIBLE answer of the Python checker to Theorem 1, and whether it covers every reason the checker gives. Here is the chain, step by step. We re-checked the Coq part for this revision with Coq 8.18: every theorem named below prints *Closed under the global context*, that is, it uses no axiom.

1. *The verdict comes from the search.* The checker answers IMPOSSIBLE with reason BLOCKED_GUARD, or with GOAL_UNSAT from the search, only when its forward search over the pack finds no goal state. MISSING_CAPABILITY is different: it is the first premise of ACH (every tool the protocol names is granted) and is decided before any search. It is a refutation in the sense of Theorem 1 only when every path to the goal needs the missing tool. While preparing this revision we found that the checker also answered IMPOSSIBLE when the missing tool sat on a branch the goal does not need, where another branch can still reach the goal. We fixed it. The checker now answers IMPOSSIBLE with this reason only when (i) every path to a goal point uses a missing tool, a check that ignores guards and so over-approximates the runs; (ii) no protocol over Γ can reach the goal at all; or (iii) the search, which treats each step with a missing tool as a dead end (WORLD-ACT cannot fire it), finds no goal state. In every other case the goal is decided by the search, and the missing tool is recorded as a protocol rejection, so a host still refuses the plan. In all our corpora every missing tool lies on the only path, so the results above are unaffected.
2. *The search is the abstraction of Lemma 3.* True/false facts are kept exactly. Numbers are kept as one accumulated constraint. An action step is taken only when the constraint and the guard can hold together; it applies the same add/delete update as the real step and replaces the constraint by the strongest postcondition. At a loop's back edge the constraint is dropped (widening). Messages and branch choices do not touch numbers: they are steps with a true guard and no numeric effect (a branch's assumed facts are an add effect), so the Coq model, which is generic in the type of steps, covers them as well.
3. *Lemma 3 holds for this abstraction* — machine-checked as `SymbolicInstance.step_sim` and `goal_sim`, for every widening policy.
4. *Theorem 1 then applies* — machine-checked as `refutation_sound_rel`, and composed with step 3 as `SymbolicInstance.symbolic_refutation_sound`.

Three points are argued on paper, not in Coq. *(a) Loop pruning.* When the search comes back to the same loop with the same true facts, it stops. After widening, the abstract state at that point depends only on the loop and the facts, so it is a state the search has already explored or is still exploring, and any path through it twice can be shortened; stopping therefore loses no reachable goal state. This needs loop names to be unique, which the schema gate enforces. *(b) Formulas.* That the checker's QF-LIA formulas mean the predicates of the Coq model; the code renames each variable per step instead of computing the postcondition symbolically. *(c) The solver.* That Z3 answers correctly; when it times out, the checker counts the query as satisfiable, which can only prevent a refutation. Other verdicts rest on other results: a protocol rejection is not a goal refutation at all (Section 4); a refutation by the establisher closure (no capability, and no initial fact, can make a goal fact true, so no run of any protocol over Γ reaches the goal) rests on that one-line argument, argued on paper, of which `FlightInstance` mechanizes one instance; the optional adversarial mode, where the environment chooses some branches, is argued on paper only; and a refutation by the requirement reader rests on a runtime manifest, not on Theorem 1. Every verdict states which of these covers it.

D.4 Operational correspondence: subject reduction and session fidelity

The direct judgment $G \vdash (\mathbb{M}; W)$ is not merely preserved by reduction: the session and its protocol step *in lockstep* over the labelled transition systems of Section B.2. This is the standard soundness backbone of a session calculus [13, 11], obtained here *directly*, with no projection and no merge.

When actions by different participants change the world, we assume that their effects commute. Communication-only reductions satisfy this condition automatically. Goal markers require no additional condition because G-GOAL-F checks the goal in the world state where the step occurs.

Lemma 4 (Residual capability)**.** *If* $(G, W) \rightsquigarrow_\Lambda (G', W')$*, then* $\Lambda \in \mathsf{cap}(G)$.

Lemma 5 (Participant agreement)**.** *If* $G \vdash (\mathbb{M}; W)$*, then* $\mathsf{prt}(G) = \mathsf{prt}(\mathbb{M})$.

Both follow by induction on the transition or typing derivation, respectively; they are the capability-membership and participant invariants used in the interleaving cases below.

Theorem 4 (Subject reduction)**.** *If* $G \vdash (\mathbb{M}; W)$ *and* $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$*, then* $(G, W) \rightsquigarrow_\Lambda (G', W')$ *for some* G' *with* $G' \vdash (\mathbb{M}'; W')$.

Theorem 5 (Session fidelity)**.** *If* $G \vdash (\mathbb{M}; W)$ *and* $(G, W) \rightsquigarrow_\Lambda (G', W')$*, then* $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$ *for some* $\mathbb{M}'$ *and* W' *with* $G' \vdash (\mathbb{M}'; W')$.

We prove both by induction on the typing derivation $G \vdash (\mathbb{M}; W)$, with a case analysis on the transition; the one auxiliary fact used throughout is that typing respects pointwise-equal sessions (`ctypes_ext`), which lets the interleaving cases reassociate independent updates.

Proof of Theorem 4 (subject reduction). By induction on $G \vdash (\mathbb{M}; W)$.

Case T-END ($G = \mathsf{End}$, every role idle). Then $\mathbb{M}$ has no output or pending action, so no session transition exists and the claim holds vacuously.

Case T-GOAL ($G = \checkmark\varphi.G_0$, with $W \vDash \varphi$ and $G_0 \vdash (\mathbb{M}; W)$, $\mathsf{prt}(G_0) = \mathsf{prt}(\mathbb{M})$). A marker is not a session construct, so the step $(\mathbb{M}, W) \to_\Lambda (\mathbb{M}', W')$ is a step of G_0's session; the induction hypothesis gives $(G_0, W) \rightsquigarrow_\Lambda (G', W')$ with $G' \vdash (\mathbb{M}'; W')$. Since $W \vDash \varphi$, rule G-GOAL-F discharges the marker and performs the same step, $(\checkmark\varphi.G_0, W) \rightsquigarrow_\Lambda (G', W')$; the residual typing is the one the induction hypothesis provides. No stability hypothesis is needed: the marker is checked at W, where the step happens.

Case T-COMM, with head $\mathsf{p}{\to}\mathsf{q}$ and branches G_i. Here $\mathbb{M}$ is $\mathsf{p}[\mathsf{q}!\dots] \parallel \mathsf{q}[\mathsf{p}?\dots] \parallel \mathbb{M}_0$, and each branch satisfies $G_i \vdash (\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}_0; W)$. The transition is a communication $\Lambda = \mathsf{r}\,\ell\,\mathsf{t}$. Because p offers only outputs to q and q only inputs from p, the sender/receiver shapes force a dichotomy.

- *Head move* ($\mathsf{r} = \mathsf{p}$). Then $\mathsf{t} = \mathsf{q}$ and $\ell = \ell_k \in I$, and $\mathbb{M}' = \mathsf{p}[P_k] \parallel \mathsf{q}[Q_k] \parallel \mathbb{M}_0$. Rule G-COMM-E gives $(G, W) \rightsquigarrow_{\mathsf{p}\ell_k\mathsf{q}} (G_k, W)$, and $G_k \vdash (\mathbb{M}'; W)$ is exactly the k-th premise.
- *Interleaving move* ($\mathsf{r} \neq \mathsf{p}$). The shapes then force $\mathsf{r}, \mathsf{t} \notin \{\mathsf{p}, \mathsf{q}\}$: r is an output so $\mathsf{r} \neq \mathsf{q}$, and t is an input so $\mathsf{t} \neq \mathsf{p}$, while $\mathsf{t} = \mathsf{q}$ would force $\mathsf{r} = \mathsf{p}$. Since r, t lie in the bystanders $\mathbb{M}_0$, the *same* communication is enabled in each branch's configuration $\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}_0$ (the head updates do not touch r, t). The induction hypothesis at each i yields G_i' with $(G_i, W) \rightsquigarrow_\Lambda (G_i', W)$ and $G_i' \vdash (\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}_0'; W)$, where $\mathbb{M}_0'$ is $\mathbb{M}_0$ after the bystander step. As $\mathsf{prt}(\Lambda) = \{\mathsf{r}, \mathsf{t}\}$ is disjoint from $\{\mathsf{p}, \mathsf{q}\}$, rule G-COMM-I assembles $(G, W) \rightsquigarrow_\Lambda (\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i'\}, W)$, and re-applying T-COMM (the sender/receiver at p, q are untouched, and each branch is retyped by the induction hypothesis, reassociating the independent updates via `ctypes_ext`) gives the residual typing.

Communications leave the world fixed, so no independence hypothesis is needed. For a world-changing action at the head (T-ACT), G-ACT-E matches and the residual G is typed at the *same* post-effect world the rule reaches — whichever successor world the session's firing picks, the universal premise of T-ACT types the residual at it. Interleaving *two* actions requires their effects to commute ($\mathsf{eff}_a \circ \mathsf{eff}_b = \mathsf{eff}_b \circ \mathsf{eff}_a$ for disjoint a, b), a frame-independence side condition that participant-disjointness alone does not supply; under it the T-ACT interleaving case goes through identically. □

Proof of Theorem 5 (session fidelity). By induction on $G \vdash (\mathbb{M}; W)$, inverting the global step $(G, W) \rightsquigarrow_\Lambda (G', W')$. T-END admits no global step in the correspondence, so the claim holds vacuously. For T-GOAL, the step is G-GOAL-F; its premise $(G_0, W) \rightsquigarrow_\Lambda (G', W')$ is realised by the induction hypothesis as a session step, which is the required one — the marker exists only

on the type side and is discharged by the same transition. For T-COMM, the global step is either G-COMM-E — and the prescribed head communication $\mathsf{p}\,\ell_k\,\mathsf{q}$ is enabled by S-COMM, landing in the k-th premise — or G-COMM-I with $\mathsf{prt}(\Lambda) \cap \{\mathsf{p}, \mathsf{q}\} = \emptyset$; then the induction hypothesis on any branch produces a bystander communication, whose endpoints lie outside $\{\mathsf{p}, \mathsf{q}\}$, so S-COMM fires it on $\mathbb{M}$ and T-COMM retypes the result. The action cases mirror those of Theorem 4. □

Mechanization. Both theorems are mechanized axiom-free for the single-label communication fragment — the fragment in which bystander interleaving is unconditional — in `DirectTypingSR.v`, over labelled session and global transition systems, with no projection, merge, or local type:

```
Theorem subject_reduction :  forall W G s s' L,
ctypes W G s -> lstep L s s' ->
exists G', gstep W L G G' /\ ctypes W G' s'.
Theorem session_fidelity :  forall W G G' s L,
ctypes W G s -> gstep W L G G' ->
exists s', lstep L s s' /\ ctypes W G' s'.
(* Print Assumptions => Closed under the global context (both).  *)
```

The head-move case for world-changing *actions* is additionally mechanized in `DirectTyping.v` (`type_directed_safety`, also `progress`). Goal labels appear in neither label alphabet: the paper's correspondence uses the same fused goal rule (G-GOAL-F) as the Coq `GS_Goal`. Participant-matching side conditions, labelled action interleaving, and multi-branch choice are still absent from the mechanization. Completing a faithful mechanization of the paper rules remains work in Section 7.

Concrete instance. `HandoffInstance` mechanizes the planner/worker handoff of Section 1 on both sides. The *good* pairing — planner sends, worker delivers, goal checked — is proved `ctypes`-typed (`good_typed`) and its run is proved to reach a world satisfying the goal (`good_runs_to_goal`). The *bad* pairing — both roles start with an input, exactly the deadlock of Section 1 — is proved `Stuck` (`bad_stuck`), and the contrapositive of `progress` then gives, for free, that *no non-trivial global type can ever type it* (`bad_untypeable`): the rule rejects the classical deadlocking handoff without any projection ever being computed.

D.5 Incompleteness, by design

Proposition 1 (Incompleteness)**.** $\Gamma; G \vDash \Diamond_{init}\varphi_{goal}$ *may hold while no concrete runtime execution reaches* φ_{goal}*: the witnessing abstract path can be spurious when* α *collapses a distinction that mattered. The system is* sound for ✗ and incomplete for ✓.

This is the price of decidable, tolerant over-approximation; tightening α trades tolerance for precision. The residue is confined to two edges, motivating the layered architecture:

- **Payload faithfulness** (bottom edge): a tool's declared post-condition ("filters to fares < 500") may be false — a runtime obligation for a deterministic monitor (Layer C), not a static one.
- **Intent fidelity** (top edge): the formalized goal may not capture what the user meant — irreducibly semantic, surfaced at the single compaction checkpoint for human review.

D.6 Decidability and the autonomy boundary

Theorem 6 (Termination and soundness of the widened search)**.** *In the fragment with finitely many roles (static topology), regular global types represented by finite tail-recursive control graphs, decidable state logic* $\mathcal{L}$*, and* $\mathcal{L}$*-definable effects, the widened symbolic search terminates on every pack. It decides the widened abstract judgment; for* $\Gamma; G \vDash \Diamond_{init}\varphi_{goal}$ *its* IMPOSSIBLE *answer is sound (Theorem 1 with Lemma 3) and its* ACHIEVABLE *answer may be spurious (Proposition 1).*

Proof. The pack-level achievability judgment $\Gamma; G \vDash \Diamond_{init}\varphi_{goal}$ (Section B.2) is an existential reachability query over configurations (G', W); we exhibit the terminating procedure and place its two answers.

The search space is finite. A configuration has two components. (i) *Control*, the residual global type G' reachable from G by $\rightsquigarrow$. Since G is a regular tree (tail-recursion: finitely many distinct

subtrees, Section A.3) and each $\rightsquigarrow$ step either descends into a subtree (G-COMM-E, G-ACT-E) or erases a marker (G-GOAL-E), the set $\{G' : (G, _) \rightsquigarrow^* (G', _)\}$ is a subset of the finitely many subtrees of G, hence finite. (ii) *Symbolic world.* The boolean part is a valuation $B : \mathsf{Pred} \to \mathbb{B}$ over the finitely many predicates named in the pack, so there are $2^{|\mathsf{Pred}|}$ boolean states. The numeric part is summarized by an accumulated constraint $\psi \in \mathcal{L}$ (a conjunction of the guards taken and the Nd-postconditions assumed); on a control-graph back edge the reference procedure applies the widening $\psi \mapsto \top$, forgetting accumulated numeric constraints. This one-element numeric summary is an over-approximation and, together with the finite boolean valuations, leaves only finitely many symbolic worlds. The product with finite control is therefore finite.

Each edge is computable. Firing a capability requires checking $W \vDash pre_a$ and forming sp; testing goal-satisfaction requires $W \vDash \varphi$. Both are entailment/satisfiability queries in quantifier-free linear integer arithmetic with uninterpreted predicates — a decidable theory, discharged by an SMT solver. Branch selection (REACH-OR) is a finite disjunction. Existence of an initial world is decided by the same SMT query over $init$; each satisfying symbolic initial valuation seeds the finite search.

Termination and correctness. Forward search (breadth-first over $\rightsquigarrow$) maintains a visited set of (control, symbolic-world) pairs; because that set is finite it adds each pair at most once, so the search halts. It answers ACHIEVABLE iff it enumerates a goal-satisfying configuration, and IMPOSSIBLE otherwise. Soundness of the IMPOSSIBLE verdict is Theorem 1 with Lemma 3; and widening only *enlarges* the reachable set (it weakens constraints, adding edges), so by Theorem 2 it never introduces a false refutation. Hence the search always terminates and decides the widened abstract judgment; IMPOSSIBLE transfers to the exact judgment, ACHIEVABLE does not (Proposition 1). □

Why not exact decidability. With unbounded integers, updates $x := x \pm 1$, and guards $x = 0$, $x \geq 1$ under recursive control, packs encode two-counter machines, so the exact judgment $\Gamma; G \vDash \Diamond_{init} \varphi_{goal}$ is undecidable even with a fixed set of participants. The widening is what buys termination; the price is the spurious-ACHIEVABLE residue of Proposition 1.

Proposition 2 (Undecidability in a dynamic-topology extension)**.** *Consider the extension G^+ of the grammar with* add r. *G, which creates (adds) a new participant during execution: the new participant runs a declared finite behaviour template and brings one fresh numeric variable, initialized by the step that adds it. With an unbounded role population and dynamic channels, achievability in G^+ is undecidable — and no widening restores termination, since the reachable control space itself becomes infinite.*

Proof sketch. By reduction from the control-state reachability problem for communicating finite-state machines (CFSMs), which is undecidable [15]. Fix a CFSM instance: finite-control machines M_1, M_2 exchanging messages over unbounded FIFO channels, and a target control state q_f of M_1; deciding whether a run reaches q_f is undecidable.

We construct a skill whose goal is achievable iff the CFSM reaches q_f. Each machine M_i becomes a role whose process follows its control graph. When a machine sends message m, the process adds a new participant carrying m. When a machine receives m, it synchronizes with the oldest outstanding participant for that channel. This unbounded set of participants represents the unbounded FIFO queue. The message order is enforced using sequence numbers: each new participant stores the current send counter, and a receive is allowed only when that number matches the receive counter. This requires a fresh variable for each participant, which is available in G^+ but not in the base fragment with a fixed Var.

The Asg/Nd machinery of Section A.2 supplies the counter updates. A boolean predicate at_q_f is established by the single capability guarding M_1's entry into q_f, and we set $\varphi_{goal} = \mathsf{at}_q_f$. By construction a run of the skill reaches a state satisfying φ_{goal} iff the CFSM has a run reaching q_f: the courier discipline then puts the skill's reachable configurations in correspondence with the CFSM's channel contents and control states. The construction is a computable map from CFSM instances to packs, so a decision procedure for achievability would decide CFSM control-state reachability — impossible. Hence achievability is undecidable once new participants may be added during execution. (The finiteness argument of Theorem 6 breaks at exactly the point this reduction exploits: an unbounded number of couriers makes the set of reachable controls infinite.) □

The two results identify a concrete boundary: when the set of participants is finite and fixed, the widened search terminates and its *impossible* verdict is sound. If an unbounded number of new participants can be added during execution, no such finite search can decide the problem in general.This result concerns only the number of participants; it does not mean that all forms of agent autonomy are undecidable. The implementation returns UNKNOWN when a pack violates the static-topology side condition of Theorem 6, unless an independent capability or establisher refutation has already proved it impossible.

D.7 The partition of obligations

Corollary 1. *The architecture separates "will the goal be achieved" into four obligations on disjoint substrates:*

Concern	Where	Mechanism
goal achievability	static, terminating	Coq specification; deterministic search; no LLM
per-step safety / least privilege	Layer B (future)	same engine, □ + permission tiers
payload faithfulness	Layer C (future), run-time	deterministic monitors with blame
intent fidelity	top edge, synthesis	one human checkpoint

D.8 Verifiable feedback beyond a scalar reward

The verdict is a heterogeneous verification signal rather than a scalar score. An IMPOSSIBLE result identifies which premise failed and carries a blocking frontier; ACHIEVABLE carries a witness path and explicitly retains the two deferred obligations; UNKNOWN is an abstention outside the decidable fragment, not a refutation. These structured outcomes can serve as cheap negative labels for automated skill or prompt search without an LLM judge, while the intent-fidelity checkpoint keeps the irreducible semantic decision with a human. We treat such search as an application of the verifier, not as a contribution evaluated here.

E All real use cases

Table 1 lists every real use case we have run, including those without labels, which Figure 3 cannot show. A rate is a percentage over the number of tests in the "Size" column unless stated. "Real" means that the skills, workflows, files or agent runs were not written for this paper. The reports behind each row are in the artifact's `docs` folder.

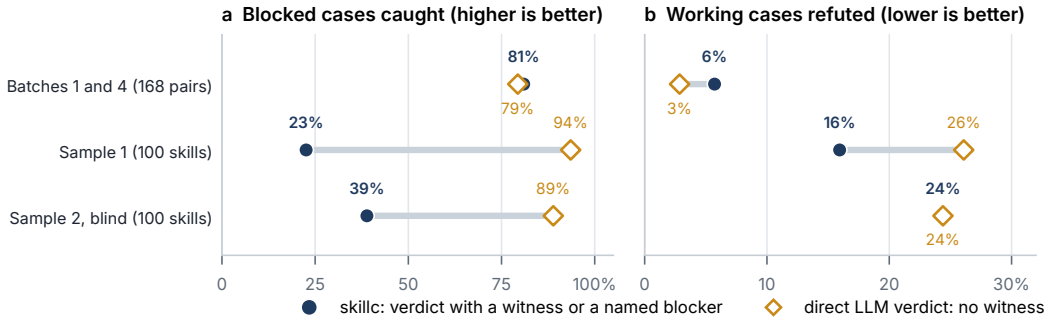


Figure 6: skillc against a direct verdict by the same model on the same cases (Claude in batches 1 and 4, gpt-5.4 in samples 1 and 2). On real skills labelled by execution the two are close; where the blocker is rarely written in the file (samples 1 and 2), the direct verdict catches more blocked files but also refutes more working ones, and gives no witness.

How reliable is compaction? The reviews of the submitted version pointed out that compaction, the untrusted step, was never measured. On sample 1, with five samples per skill, the schema gate accepted 68% over 500 first replies and 92% after one retry; all five samples agreed on 39% over 100

	Use case	Size	How truth is known	skillc result
A	Pre-flight in a real workspace	7 workflows, 21 runs	probed prerequisites	correct 100%; blocked stopped early 100% over 12; 0 tokens
A	Microsoft Agent 365 workflows	10 workflows, 123 checks	published workflows	expected result 100%; denied calls reaching a tool 0% over 50
A	Public skills in two runtimes	42 skills × 2	runtime tool lists	achievable in claude.ai 100%; refuted in Claude Code 14% (6), each naming a missing tool
B	Batch 1 of public skills (artifact: held-out and fresh)	74 pairs	blind execution	correct 85% (direct LLM 91%)
B	Batch 2 (artifact: ext)	115 pairs	blind execution	correct 90%; caught 94% over 35
B	Batch 3, three runtimes (artifact: div)	164 pairs	blind execution	correct 86%; caught 81% over 47
B	Batch 4 (artifact: gr)	94 pairs	blind execution	correct 93% (direct LLM 90%); refuted 5% over 59
C	Sample 1: SKILL.md and agent files (artifact: set2)	100 files	AI annotators, read-only probes	correct 60% over 100; caught 23% over 31
C	Sample 2: new files (artifact: set3)	100 files	AI annotators, runtime description only	correct 64% over 100; caught 39% over 18
C	Microsoft Scout skills	70 SKILL.md	none	valid pack 96%; achievable 69%, impossible 27%
C	Agent 365 workflows, compacted	195 checks	published workflows	expected result 97% with the environment's state names; 0% without
C	Hosted finance agent, compacted	9 configurations	all are correct	wrongly refuted 67%
C	Token cost	62 files of sample 1	metered model calls	saving 1.58× on blocked files; ungated agent claimed success on 32% over 31
D	Hosted agent gate, policy removed	21 runs per arm	logged sends	unapproved sends 0% (ungated 10%)
D	Runtime guard on reasoning	37 hazards, 97 benign	blind incident traces	caught 27%; benign flagged 2%
D	Runtime monitor, headless Claude Code	4 scenarios	scripted	as intended in all 4: approved a feasible plan, refused an impossible deploy and a prohibited push, revoked a plan when a program was missing

Table 1: Every real use case, by category: A, no LLM in the loop; B, real skills with LLM compaction, labelled by blind execution; C, real files or workflows with LLM compaction, labelled by AI annotators, by the workflows' published expectations, or not at all; D, watching a running agent.

skills. A separate Claude judge found no invented tool in 91% and no invented ordering in 90% of packs, but an omitted mandatory step in 37%.

F Token economics of pre-execution refutation

F.1 Measured

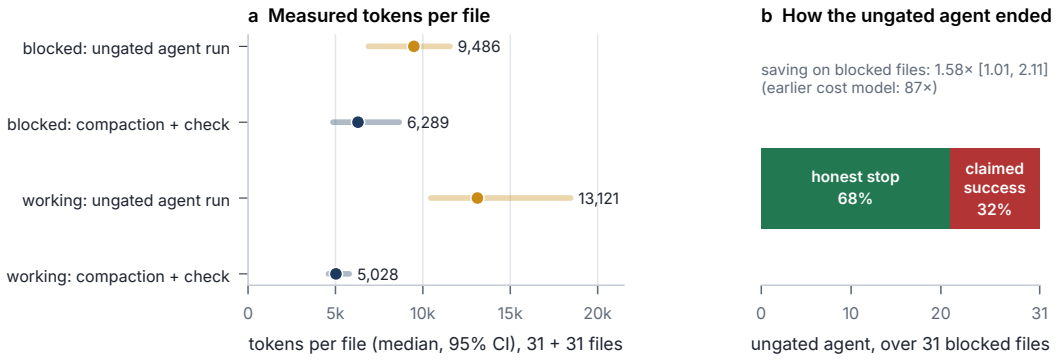


Figure 7: Measured cost on 62 files of sample 1 (31 that cannot run, 31 that can), gpt-5.4, real model calls with tool results simulated from the AI annotators' probe-verified observations. (a) Median tokens per file and 95% confidence interval. (b) How the ungated agent ended on the 31 files that cannot run.

We ran an ungated gpt-5.4 agent on 31 files that cannot run and 31 that can, and metered one compaction-plus-check per file (Figure 7). On the blocked files the ungated run used a median of 9,486 tokens and compaction used 6,289: the check saves $1.58\times$ (95% CI 1.01–2.11), where the model below predicts $51\times$ for these files and $87\times$ for the corpus. The reason is simple: the agent stopped or declared success after a median of two turns, so the quadratic term of the model never grew. On working files the check costs 0.36 of one run (95% CI 0.29–0.45), not the 3–4% of the model. Two things the token count does not show: 10 of the 31 blocked runs ended with the agent reporting success on a task that could not be done, and 7 of the 31 working runs stopped although the task was feasible. The rest of this appendix is the original cost model. We keep it because its ordering of failure modes is useful, but its absolute numbers are a model, not a measurement.

F.2 A model, not a measurement

The practical case for deciding achievability *before* execution is an economic one, so we state it in tokens rather than adjectives. Two asymmetries carry it. First, no model sits in the trusted path: the schema gate, direct conformance and Z3 reachability spend **zero tokens**, and the deterministic front-end spends zero as well. Only the optional LLM compaction spends tokens, and it is paid *once per skill version*. Second, an agent turn re-sends its whole context: writing S for the harness prompt, K for the loaded skill, g for the mean tokens appended per turn and o for output per turn, a run of T turns reads $T(S{+}K) + g\,T(T{-}1)/2$ input and To output tokens. Compaction is linear in the skill and paid once; a doomed run is *quadratic* in its turns and recurs on every invocation.

Runtime waste is necessarily a model — it prices a run that, if the refutation is correct, never happens — so we publish the model rather than a single number. Each refutation reason carries a (low, typical, high) band for how long an agent flails before that failure stops it, and how many agents are billed. With conservative defaults ($S{=}2500$, $g{=}700$, $o{=}250$) and a median real consumer skill ($K \approx 1.7$K tokens), compaction costs $\sim$2.8K tokens against the following avoided waste per prevented invocation:

Verdict reason	turns (lo–typ–hi)	waste/run	leverage
MISSING_CAPABILITY	3–8–14, 1 agent	50 240	18×
GOAL_UNSAT	8–18–30, 1 agent	176 040	63×
NON_CONFORMANT	6–15–30, 2 agents	261 900	94×
NON_PROJECTABLE	6–15–30, 2 agents	261 900	94×
BLOCKED_GUARD	12–25–50, 1 agent	305 750	110×

The ordering is the robust part, and it is the expected one. MISSING_CAPABILITY is cheapest at run time: the agent calls a tool that is not there and learns immediately, and most of the cost is the improvisation that follows. BLOCKED_GUARD is dearest, because a precondition no run can satisfy is precisely the case an agent cannot learn from — it retries to the harness turn cap. The two multi-party reasons bill two contexts. GOAL_UNSAT additionally carries a cost the token bill does not show: a run whose goal conjunct has no establisher can terminate *believing* it succeeded. DYNAMIC_TOPOLOGY claims no savings at all, because an abstention prevents nothing.

(Modelled, for the corpus as labelled in the submitted version, where one of the seven refutations was a protocol rejection that is no longer counted as a refutation.) Over the 15-spec corpus the seven structural refutations cost 12.2K tokens of compaction and avoid $\sim$1.07M tokens per round of invocations (band 0.28M–3.15M): $\sim 87\times$ leverage, or nothing at all on the deterministic path. Break-even therefore falls *inside* the first prevented run in every mode — between 0.9% and 5.5% of it. The honest denominator for a healthy skill, where the check buys nothing and still costs something, is 2.9% of one successful run over the corpus and 4.0% for a median real skill; with the deterministic front-end, zero. Prompt caching changes the price of the wasted tokens, not their number. Halving every waste default leaves leverage at $9\times$–$55\times$ and the ordering unchanged. The model, its defaults and their justification are in the artifact (`skillc.tokens`, `skillc cost`); the Coq development costs no tokens and is amortized over every skill ever checked.

NeurIPS Paper Checklist

1. **Claims**

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state that refutation is sound relative to the declared pack, distinguish UNKNOWN from refutation, and identify the mechanized and paper-only proof fragments.

2. **Limitations**

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The Limitations and Future Work section discusses relative soundness, static topology, abstraction coarseness, mechanization gaps, low recall on real files with LLM compaction, and the size of the held-out sets.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Appendix D states the simulation, stability, commutativity, finiteness, and topology assumptions with proofs or proof sketches; the artifact checks the explicitly identified Coq fragments.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 4–5 describe the deterministic checker, verdict semantics, corpus construction, binary accounting, and separate treatment of abstentions.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The public artifact contains the checker, synthetic corpora, proof developments, pinned continuous-integration workflow, and reproduction commands, plus the evaluation sets with their labels and the reports of the model runs.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: No model is trained. Section 5 names the model (gpt-5.4), the number of compaction samples per file, the sets (set2 for development, set3 held out and labelled blind) and the baselines, and reports where the results fall short.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The checker is deterministic, so the hand-written corpus has no sampling variance. The model-based results report counts with their denominators and 95% confidence intervals (Figures 3 and 7), and agreement across samples.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The checker runs on a CPU (a median of 32 ms per decision in Q2) with a 10-second solver timeout per query. Model runs use hosted gpt-5.4; their token counts are reported in Section 5 and Appendix F.

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The work uses synthetic packs and public skill and agent files, no personal data or human subjects, and makes bounded claims with explicit trust assumptions and limitations. Benchmark files are read as text and never executed.

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Broader impact paragraph discusses reduced wasted execution as well as false confidence from inaccurate packs and the human/runtime mitigations required.

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The artifact releases no trained model, scraped dataset, or other high-risk asset; it contains a deterministic checker and synthetic examples.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The real files come from public repositories and are pinned by commit and sha256 in the artifact; they are used only as test inputs. The models are used through their providers' services. Prior methods and software foundations are credited through citations.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The supplemental artifact documents the checker interface, pack schema, synthetic corpora, proof scope, limitations, and reproduction workflow.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The research involves neither crowdsourcing nor human-subject experiments.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The research involves no study participants or human-subject data.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are part of the evaluated pipeline: gpt-5.4 compacts files into packs and gives the direct-verdict baseline, DeepSeek-V4-Pro is a second model family, and a Claude model judges pack fidelity (Section 5). The paper's core method does not depend on an LLM. The main contributions—the achievability type discipline, its formal metatheory, and the deterministic checker—are defined and verified independently of LLM inference. The LLM is used only as a front-end to compact a natural-language skill into the formal pack consumed by the checker; the same formal pack can be provided directly without an LLM. The formal guarantees therefore do not depend on the behavior of any particular LLM; the measured recall and false-refutation rates of the full pipeline do.